


**Project**

**ITCS212 Web Programming**

**Submitted By**

6388170 Yunduo Luo

6388166 Lingrui Dong

6388177 Chusong Xu

**Submitted to**

Dr. Wudhichart Sawangphol

Dr. Jidapa Kraisangka

Dr. Pisit Praiwattana

Faculty of Information and Communication Technology

Mahidol University

Semester 2/2022

# CONTENTS

**Project Overview.....2**

**Navigation Diagram.....3**

**Web Application.....4**

**Web Services.....26**

**Web Services Testing Results.....59**

## Project Overview

Our goal is to create a used electronics trading platform that focuses on providing users with high quality phones, tablets and watches. To help users make informed purchasing decisions, we will provide detailed inspection reports for each product, including assessments of performance, appearance and functionality. With these reports, users can choose the best product for them based on their needs and budget.

To ensure effective management of the platform, we will implement a fully functional administrator system. Administrators can add, delete and modify other administrators' information by logging into their own accounts. In addition, administrators will be able to update and manage product information, including adding new products, deleting old products and modifying details of existing products.

By providing users with comprehensive product information and efficient management functions, we hope to create a reliable and transparent environment for trading used electronic products, so that users can easily and securely purchase the products they want.

## Navigation Diagram

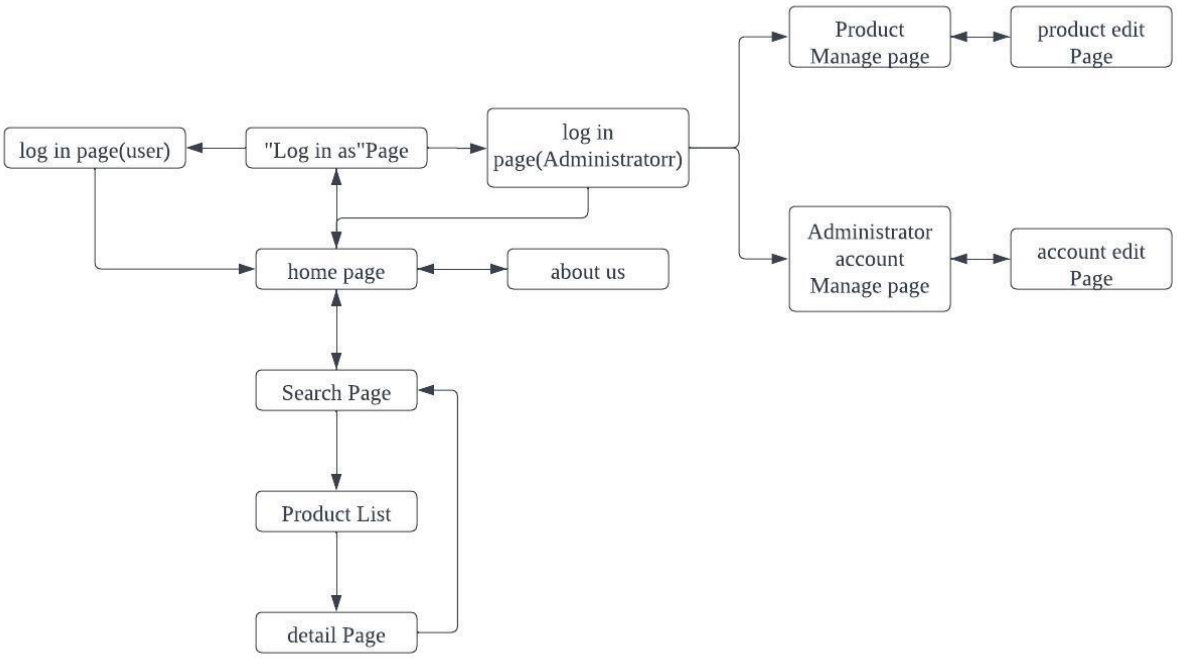



*Fig 0. Navigation Diagram*

This Navigation diagram shows the connections between the various pages. The user first enters the Login as screen, where they can choose which identity they want to use to login to our site. If the user chooses to log in as "user". They will be able to access our "Home Page" and use our main features. Click on "about us" to view the website developer information and "Search" to enter the product search screen. After a successful product search, click on the product table listed on the website and click on detail to enter the product details screen to view the product price, product details and make a purchase. Users can return to the search interface at any time by clicking the "Search" button in the navigation bar. If you are logged in as "Administrator", you will have additional administrative functions. The administrative functions are divided into Manage Products and Manage Accounts. The Administrator can modify the information in the back-end database through these two pages.

## Web Application

### HomePage

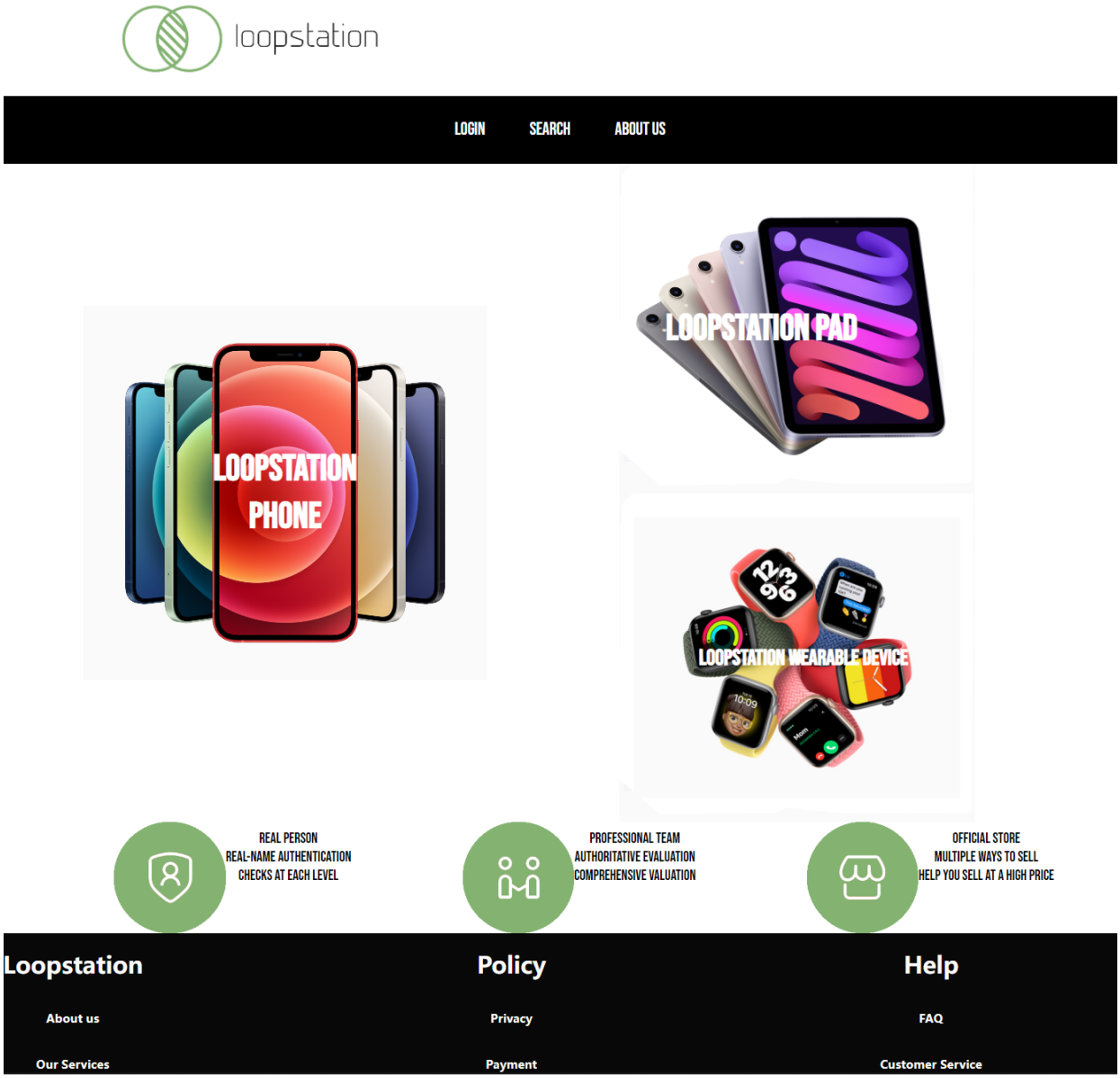



*Fig 1. Showing home page of Loopstation*

The home page is running at the root of the front service which is localhost:3031. In the home page users are able to use the navigation bar to the login page, search page and about us page. The home page displays our three main second hand products at Loopstation, phones, pad

and wearable device. It also displays the three main features of our Loopstation including real person, professional team, official store.

```
</head>

<body>
    <header>
        <img class="logo_homepage" src="/image/logo.png">
    </header>
    <nav>

        <a href="login"><i class="LoginPage"></i> Login</a>
        <a href="search"><i class="SearchPage"></i> Search</a>
        <a href="Aboutus"><i class="AboutUsPage"></i> About Us</a>

    </nav>

    <div class="container">

        <div class="left">
            <a href="#">
                <img class="phone_homepage" src="/image/phone.jpg">
                <span class="image-text-phone_homepage">Loopstation Phone</span>
            </a>
        </div>

        <div class="right">
            <a href="#">
                <img class="pad_homepage" src="/image/pad.jpg">
                <span class="image-text-pad_homepage">Loopstation Pad</span>
            </a>
            <a href="#">
                <img class="band_homepage" src="/image/band.jpg">
                <span class="image-text-band_homepage">Loopstation Wearable device</span>
            </a>
        </div>
    </div>
    <div class="introduction_image">
        <img class="RealPerson" src="/image/RealPerson.png">
        <p class="RealPerson_text">Real Person<br>Real-name authentication<br>Checks at each level</p>

        <img class="Trustworthy" src="/image/Trustworthy.png">
        <p class="Trustworthy_text">Professional Team<br> Authoritative Evaluation<br>Comprehensive Valuation</p>

        <img class="HelpYouSell" src="/image/HelpYouSell.png">
        <p class="HelpYouSell_text">Official Store<br>Multiple ways to sell<br>Help you sell at a high price</p>
    </div>

    <footer>
        <div class="column">
            <h1>Loopstation</h1>
            <a href="Aboutus"><i class="AboutUsPage"></i>
                <h4>About us</h4>
            </a>
            <h4>Our Services</h4>
            <h4>History</h4>
        </div>

        <div class="column">
            <h1>Policy</h1>
            <h4>Privacy</h4>
            <h4>Payment</h4>
            <h4>Return</h4>
        </div>
        <div class="column">
            <h1>Help</h1>
            <h4>FAQ</h3>
            <h4>Customer Service</h4>
            <h4>Order</h4>
        </div>
    </footer>

```

*Fig 2. home page html code(homePage.html)*

This is the html code of the home page. This includes four sections header, nav, body and footer. In the header, we use our Loopstation logo as the header of the home page. For the code of the navigation, we use href to link the home page to the login, search and about us page. In the body we design a container used to store the three images of phone, pad and wearable device. And in this container we provide left and right to store phone images on the left and pad and wearable device images on the right. And use introduct_image to show the three main features under the product images. In the footerm, we use three columns to display three columns of footer.

**About Page**

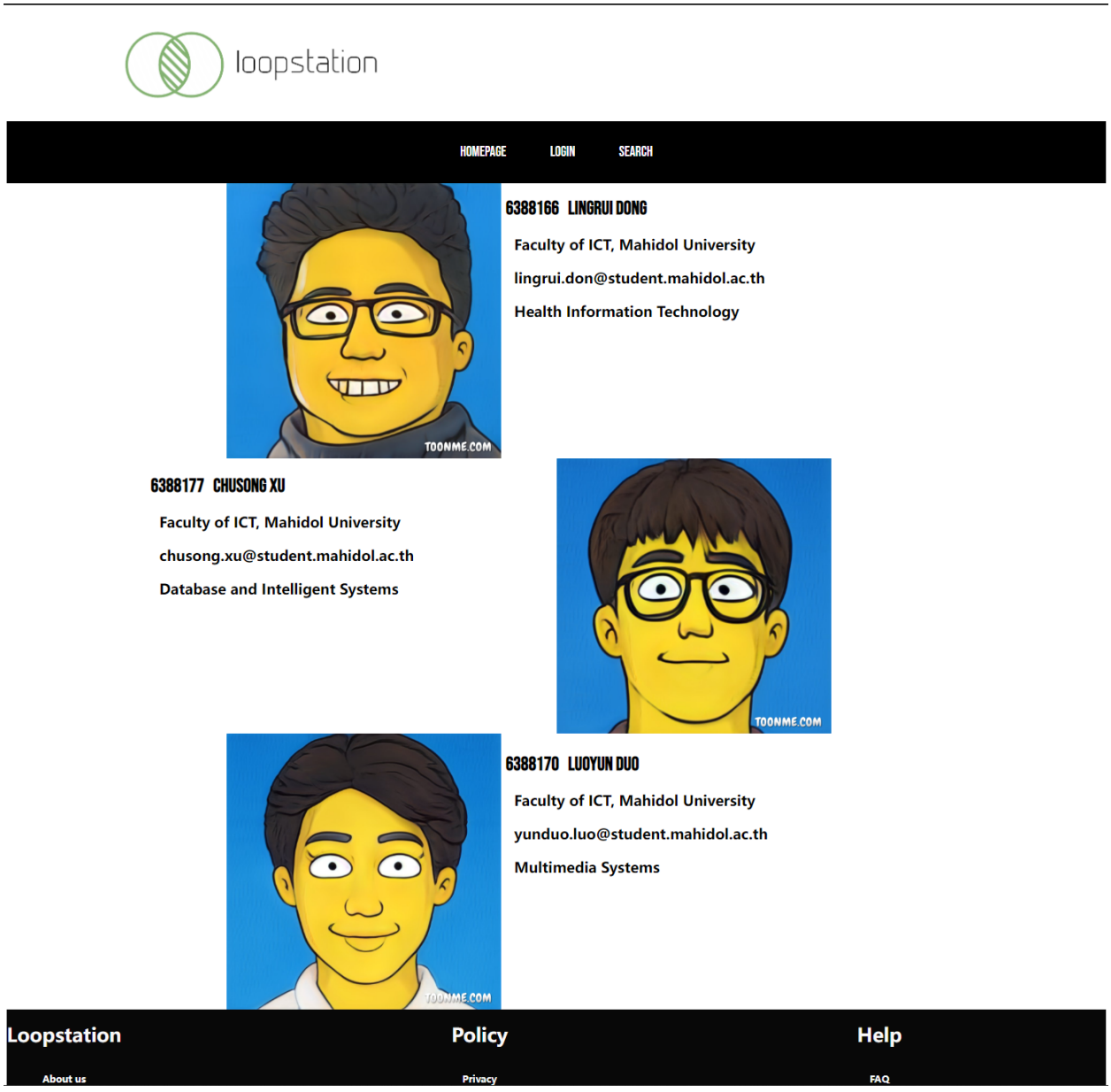



*Fig 3. Showing about us*

The about us page is running at the front service which is http://localhost:3031/Aboutus. In the about us page users are able to use the navigation bar to the login page, search page and home page. This page is used to display the basic information of each member such as student number, name, contact information, track, etc. We also used Toonme to create a portrait of each member.

```
<body>
    <header>
        <img class="logo_homepage" src="image\logo.png">
    </header>
    <nav>
        <a href="homePage"><i class="Homepage"></i> Homepage</a>
        <a href="login"><i class="LoginPage"></i>Login</a>
        <a href="search"><i class="SearchPage"></i> Search</a>

    </nav>

    <div class="image">
        <img class="LingruiDong" src="image\Lingrui Dong.jpg">
        <div class="LingruiDong_text">
            <div>
                <h1>6388166   Lingrui Dong</h1>
            </div>
            <div>
                <h3>  Faculty of ICT, Mahidol University</h3>
            </div>

            <div>
                <h3><a href="mailto:lingrui.don@student.mahidol.ac.th">  lingrui.don@student.mahidol.ac.th</a></h3>
            </div>
            <div>
                <h3>  Health Information Technology</h3>
            </div>
        </div>
        <img class="ChusongXu" src="image\Chusong Xu.jpg">
        <div class="ChusongXu_text">
            <div>
                <h1>6388177   Chusong Xu</h1>
            </div>
            <div>
                <h3>  Faculty of ICT, Mahidol University</h3>
            </div>

            <div>
                <h3><a href="mailto:lingrui.don@student.mahidol.ac.th">  chusong.xu@student.mahidol.ac.th</a></h3>
            </div>
            <div>
                <h3>  Database and Intelligent Systems</h3>
            </div>
        </div>
        <img class="LuoYunLuo" src="image\LuoYun Luo.jpg">
        <div class="LuoYunLuo_text">
            <div>
                <h1>6388170   LuoYun Duo</h1>
            </div>
            <div>
                <h3>  Faculty of ICT, Mahidol University</h3>
            </div>

            <div>
                <h3><a href="mailto:lingrui.don@student.mahidol.ac.th">  yunduo.luo@student.mahidol.ac.th</a></h3>
            </div>
            <div>
                <h3>  Multimedia Systems </h3>
            </div>
        </div>
    </div>
```

*Fig 4. About us html code(Aboutus.html)*

This is the html code of the about us page. This includes four sections header, nav, body and footer. The header and footer is the same as the home page. For the code of the navigation, we use href to link the about us page to the home page, search page and login page. In our html code, we use a class called “image” as a root class of all content. And use each team member’s name as a class to show the information of each member.

**Product Detail Page**

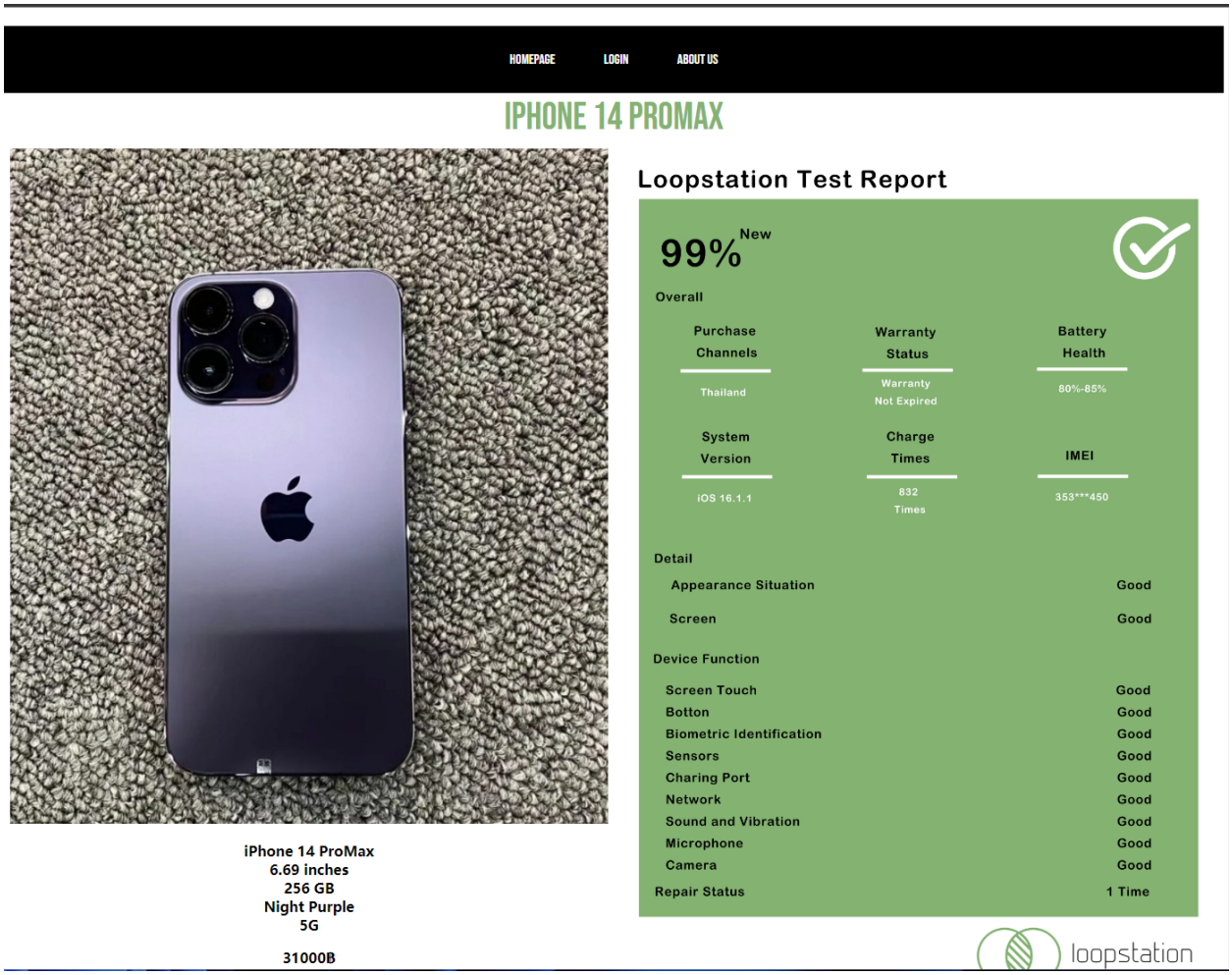



*Fig 5. Showing product detail*

The about us page is running at the front service which is http://localhost:3031/100001.html. Product detail page will show after user or administrator search the product in the search page or administer product management. In the product detail

page users are able to use the navigation bar to the login page, search page and home page. On the left side it will show the product photo, basic information about the product, etc., and on the right side it will show the Loopstation report about this product.

```
<body>
  <header>
    <img class="logo_homepage" src="image/logo.png">
  </header>
  <nav>
    <a href="homePage"><i class="Homepage"></i> Homepage</a>
    <a href="login"><i class="LoginPage"></i>Login</a>
    <a href="Aboutus"><i class="AboutUsPage"></i> About Us</a>
  </nav>
  <div class="Products_title">
    Loopstation Phone
  </div>
  <div class="Products_introduction">
    <span id="pd_rating"><br></span>
  </div>

  <div class="image-container">
    <div class="left">
      <img id="left-image" src="image/14PM.jpg">
      <p>
        <h3>
          <span id="pd_name"></span><br>
          <span id="pd_inches"></span><br>
          <span id="pd_memory"></span><br>
          <span id="pd_color"></span><br>
          <span id="pd_internet"></span><br>
        </h3>
        <h1 id="pd_price"></h1>
        <div id="currency-selection">
          <select id="currency-dropdown">
            <option value="THB">THB</option>
            <option value="JPY">JPY</option>
            <option value="USD">USD</option>
            <option value="EUR">EUR</option>
            <option value="CNY">CNY</option>
          </select>
        </div>
        <button onclick="href='#'">BUY</button>
      </p>
    </div>

    <div class="right">
      <img id="right-image" src="image/9914PM.jpg">
    </div>
  </div>

  <footer>

    <div class="column">
      <h1>Loopstation</h1>
      <a href="Aboutus"><i class="AboutUsPage"></i><h4>About us</h4></a>
      <h4>Our Services</h4>
      <h4>History</h4>
    </div>

    <div class="column">
      <h1>Policy</h1>
      <h4>Privacy</h4>
      <h4>Payment</h4>
      <h4>Return</h4>
    </div>
    <div class="column">
      <h1>Help</h1>
      <h4>FAQ</h4>
      <h4>Customer Service</h4>
      <h4>Order</h4>
    </div>
  </footer>

  <script src="javascript/productDetail.js"></script>
```

*Fig 6. Product detail html code(10001.html)*

This is the html code of the product detail page. This includes four sections header, nav, body and footer. The header and footer is the same as the home page. For the code of the

navigation, we use href to link the about us page to the home page, search page and login page. And in the image-container there are two parts which are the left and right. In the left part it uses img to read the product image and span to get the product detail information from the database. Then that has an option to change the exchange rate, which allows the user to implement a currency change for the product price based on the API of the real-time exchange rate.

**Login Page**

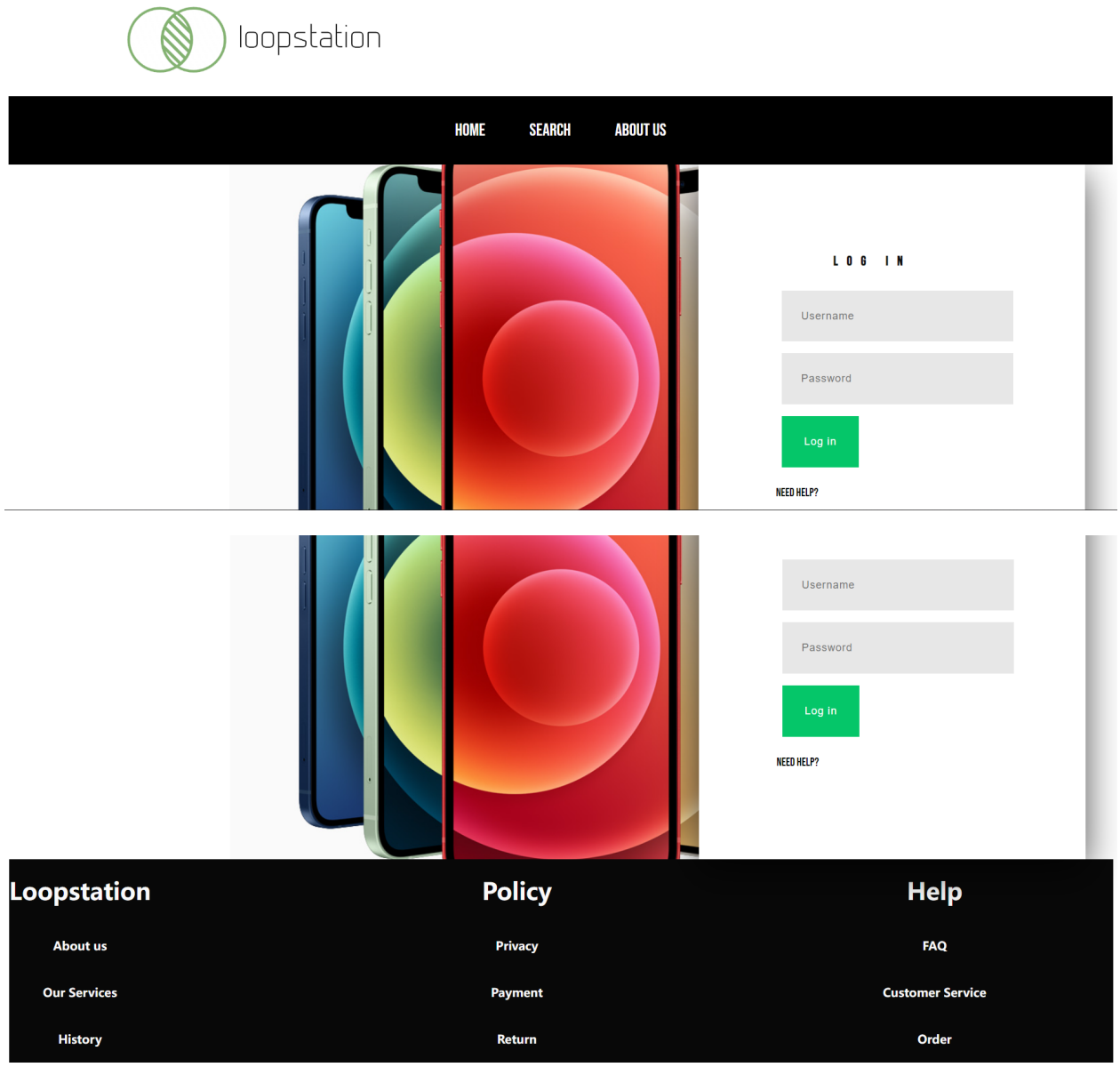



*Fig 7. Showing Login page of Loopstation*

The login page is running at the root of the front service which is localhost:3031. On the Login page you can login with administrator accounts, and then you will log into the usermanagement page.

```
<!DOCTYPE html>
<html lang="en">
<title>Loopstation login</title>

<head>
    <link rel="stylesheet" href="css/lgincs.css">
    <link rel="stylesheet" href="css/Homepage.css">
</head>

<body>
    <header>
        <img class="logo_homepage" src="image/logo.png" />
    </header>

            <nav>

                <a href="homePage"><i class="HomePage"></i>Home</a>
                <a href="search"><i class="SearchPage"></i> Search</a>
                <a href="Aboutus"><i class="AboutUsPage"></i> About Us</a>
            </nav>

            <section>
                <div class="container">
                    <div class="user">
                        <div class="formBx">
                            <form action="#" id="loginForm" onsubmit="event.preventDefault();">
                                <h2>Log in</h2>
                                <input type="text" id="log_id" placeholder="Username">
                                <input type="password" id="password" placeholder="Password">
                                <input type="submit" id="button" value="Log in">
                                <p>Need help?</p>

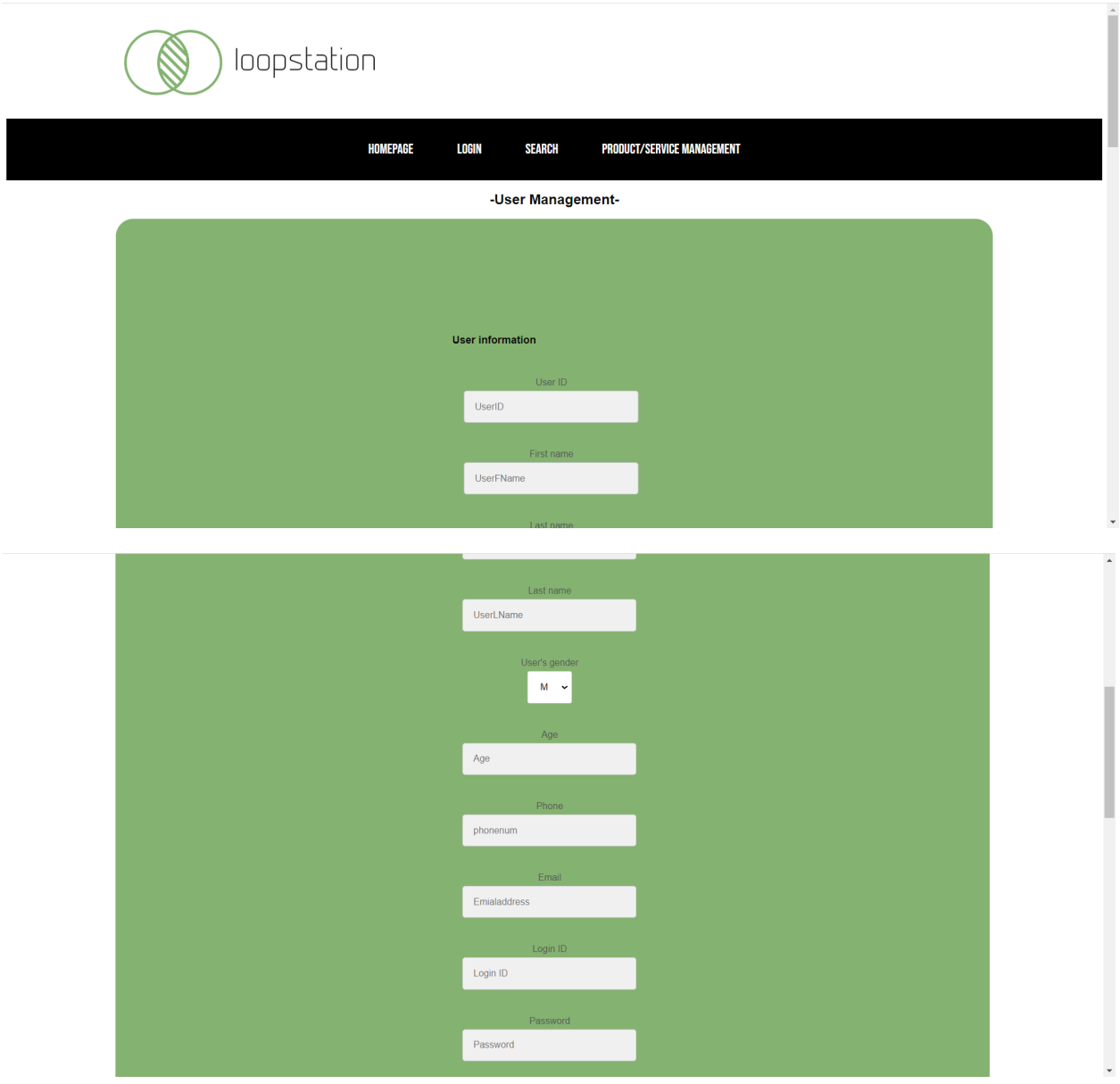
                            </form>
                        </div>
                    </div>
            </section>

    <footer>
        <div class="column">
            <h1>Loopstation</h1>
            <a href="Aboutus"><i class="AboutUsPage"></i>
                <h4>About us</h4>
            </a>
            <h4>Our Services</h4>
            <h4>History</h4>
        </div>
```

*Fig 8. Login page html Code*

The login page use the ‘herf’ to connect homepage, search page and About us page, it also connect to lgincs.css to be the style sheet, it create the form and button for administrator to login to the web services.

**User management Page**



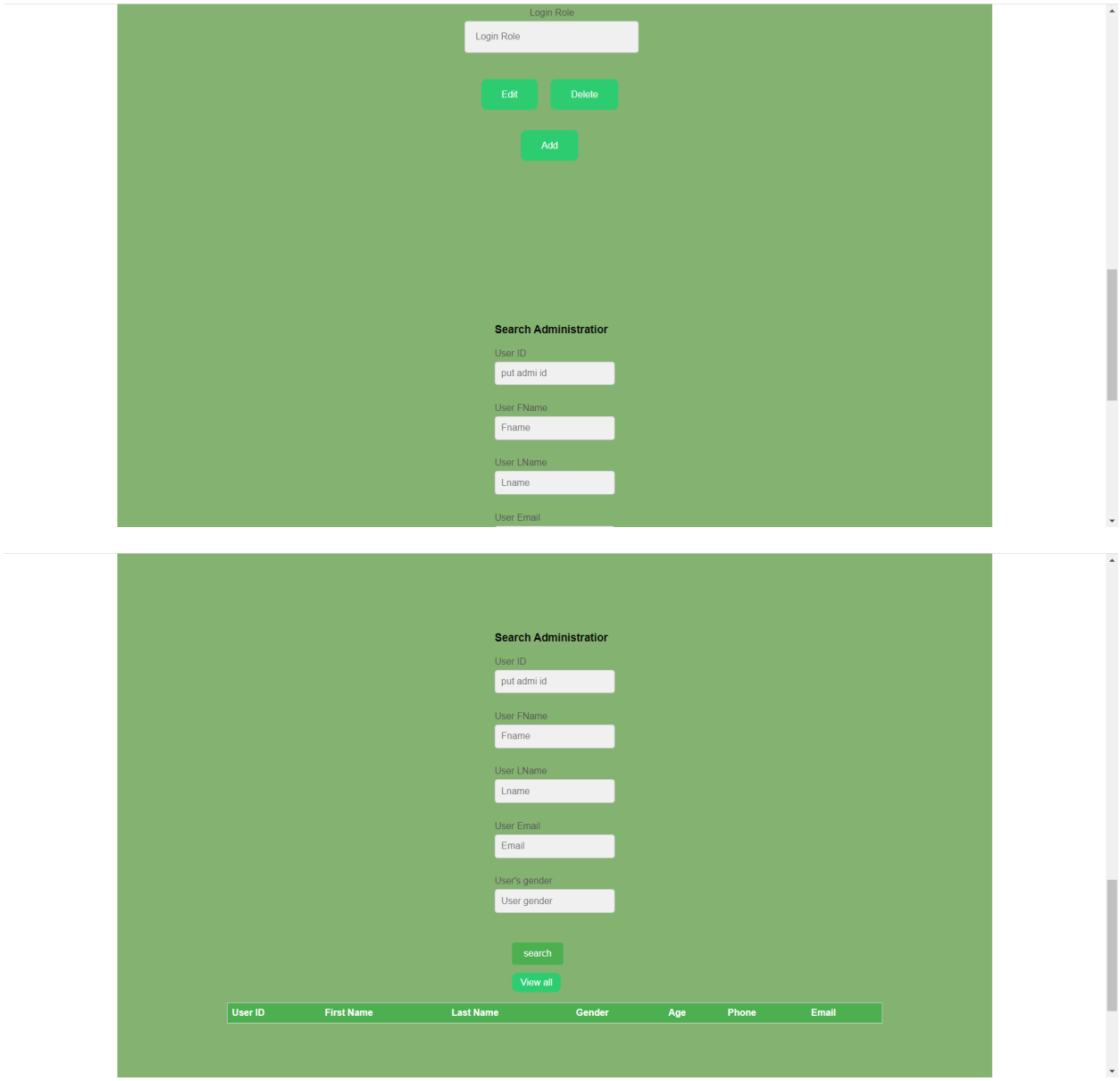



*Fig 9. User Management page*

The User Management page is running at the root of the front service which is localhost:3031. This page provide the service for administrator to modify the other administrator information. And it provide three function to modify the administrator information, Edit, Delete and Add, also can search administrator information by search by detail information and can view all administrator information.

```
<div class="title">
    <h2>-User Management-</h2>
</div>
<div class="form">
    <form class="type">
        <div>
            <h3>User information</h3>
        </div>
        <div class="item">
            <div class="column-lab">
                <label for="username">User ID</label>
                <input type="text" id="userID" placeholder="UserID">
            </div>
            <div class="column-lab">
                <label for="userfname">First name</label>
                <input type="text" id="userFname" placeholder="UserFName">
            </div>
            <div class="column-lab">
                <label for="userlname">Last name</label>
                <input type="text" id="userLname" placeholder="UserLName">
            </div>
            <div class="column-lab">
                <label for="usermale">User's gender</label>
            </div>
                <div class="column-lab">
                    <select id="Gender" type="text" required>
                        <option value="M">M</option>
                        <option value="F">F</option>
                    </select>
            </div>
            <div class="column-lab">
                <label for="userage">Age</label>
                <input type="text" id="Age" placeholder="Age">
            </div>
            <div class="column-lab">
                <label for="userPhone">Phone</label>
                <input type="text" id="Phone" placeholder="phonenum">
            </div>
            <div class="column-lab">
                <label for="useremail">Email</label>
                <input type="text" id="Email" placeholder="Emialaddress">
            </div>
            <div class="column-lab">
                <label for="loginId">Login ID</label>
                <input type="text" id="loginId" placeholder="Login ID">
```

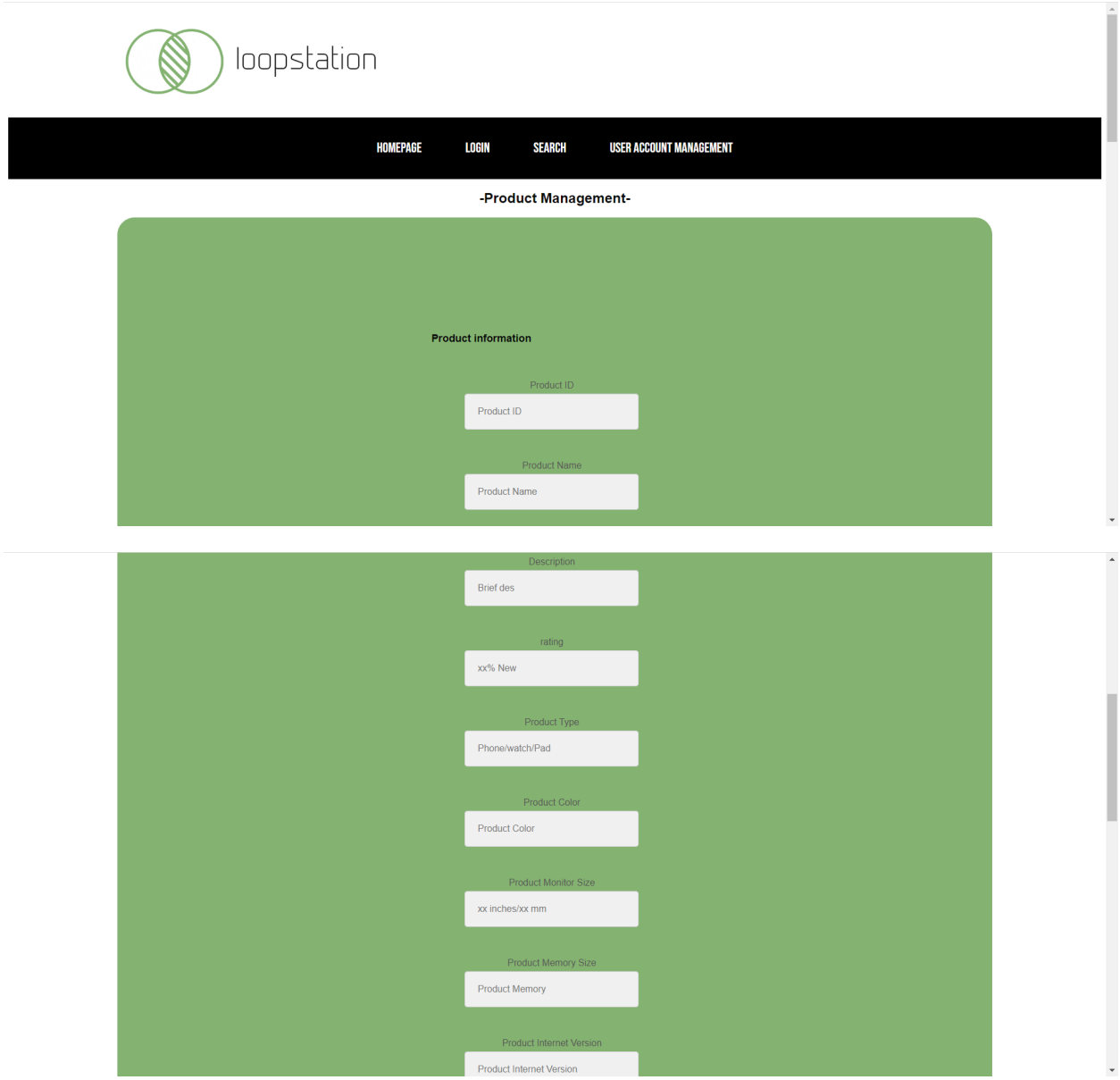
```
            </div>
            <div class="column-lab">
                <label for="password">Password</label>
                <input type="text" id="password" placeholder="Password">
            </div>
            <div class="column-lab">
                <label for="loginRole">Login Role</label>
                <input type="text" id="loginRole" placeholder="Login Role">
            </div>
            <div class="bottem">
                <input type="button" id="edit_u" value="Edit">
                <input type="button" id="deleteItem_u" value="Delete">
                <input type="button" id="add_u" value="Add">
            </div>
        </div>
    </form>
    <!--Search Admi infor-->
    <form id="Search admi">
        <div class="item">
            <div>
                <h3>Search Administratior</h3>
            </div>
            <div class="col-lab">
                <label for="searchID">User ID</label>
                <input type="text" id="searchid" placeholder="put admi id" required>
            </div>
            <div class="col-lab">
                <label for="searchName">User FName</label>
                <input type="text" id="search_fname" placeholder="Fname" required>
            </div>
            <div class="col-lab">
                <label for="searchName">User LName</label>
                <input type="text" id="search_lname" placeholder="Lname" required>
            </div>
            <div class="col-lab">
                <label for="searchEmail">User Email</label>
                <input type="text" id="search_email" placeholder="Email" required>
            </div>
            <div class="col-lab">
                <label for="searchGender">User's gender</label>
                <input type="text" id="searchgender" placeholder="User gender" required>
            </div>
        </div>
    </form>
```

```
    <div class="bottem">
        <input type="submit" id="searchUsers" value="search">
        <input type="button" id="viewall_u" value="View all">
    </div>
    <br>
    <!-- Search result table -->
    <table>
        <thead>
            <tr>
                <th>User ID</th>
                <th>First Name</th>
                <th>Last Name</th>
                <th>Gender</th>
                <th>Age</th>
                <th>Phone</th>
                <th>Email</th>
            </tr>
        </thead>
        <tbody id="admi-list">
            <!-- Search result will be added here using JavaScript -->
        </tbody>
    </table>
    </table>
</div>
```

*Fig 10. User Management page html code*

This is a form for a user management system that contains input and search functions for user information, as well as a form for displaying search results. The form contains input boxes for user ID, name, gender, age, phone, email, login ID, password, login role and other information, which can be edited, deleted, added and other operations. There is also a form for searching administrators, which can be searched by user ID, name, email, gender, etc. The final results are displayed in a form.

## Product management Page



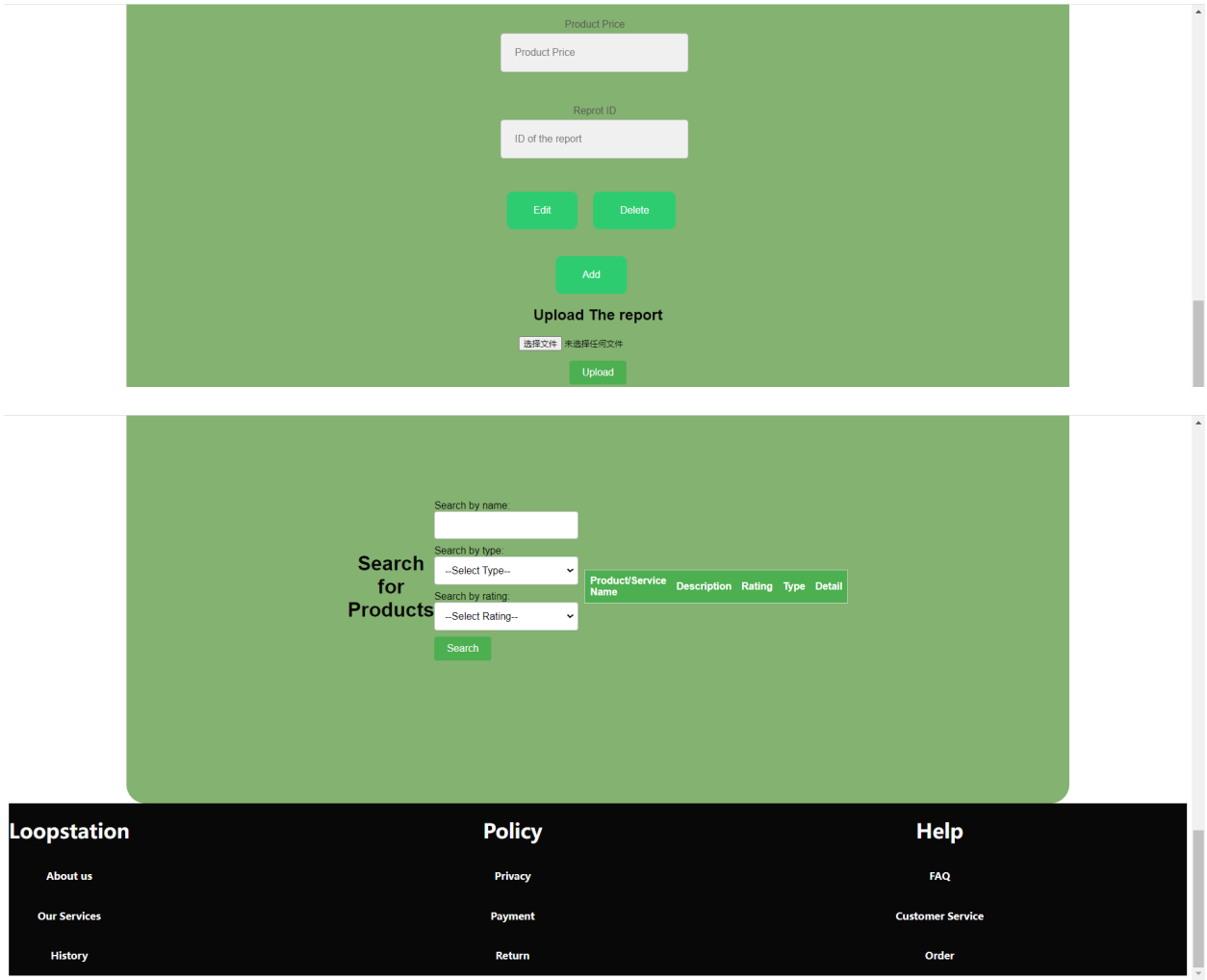



*Fig 11. Product Management page*

This page is for administrator to manage the product, and is running at the root of the front service which is localhost:3031. And it will allow administrator to Edit, Add and Delete the products, administrator also can search the products by name, type and rating

```
<!--edit the product-->
<div class="title">
    <h2>-Product Management-</h2>
</div>
<div class="form">
    <form class="type">
        <div>
            <h3>Product information</h3>
        </div>
        <div class="item">
            <div class="column-lab">
                <label for="pID">Product ID</label>
                <input type="text" id="productID" placeholder="Product ID">
            </div>
            <div class="column-lab">
                <label for="pName">Product Name</label>
                <input type="text" id="productName" placeholder="Product Name">
            </div>
            <div class="column-lab">
                <label for="pDescript">Description</label>
                <input type="text" id="des" placeholder="Brief des">
            </div>
            <div class="column-lab">
                <label for="pRate">rating</label>
                <input type="text" id="rate" placeholder="xx% New">
            </div>
            <div class="column-lab">
                <label for="pType">Product Type</label>
                <input type="text" id="ptype" placeholder="Phone/watch/Pad">
            </div>
            <div class="column-lab">
                <label for="pColor">Product Color</label>
                <input type="text" id="color" placeholder="Product Color">
            </div>
            <div class="column-lab">
                <label for="pInche">Product Monitor Size</label>
                <input type="text" id="inche" placeholder="xx inches/xx mm">
            </div>
            <div class="column-lab">
                <label for="pMemory">Product Memory Size</label>
                <input type="text" id="memory" placeholder="Product Memory">
            </div>
            <div class="column-lab">
                <label for="pInternet">Product Internet Version</label>
                <input type="text" id="internet" placeholder="Product Internet Version">
            </div>
```

```
            <div class="column-lab">
                <label for="pPrice">Product Price</label>
                <input type="text" id="price" placeholder="Product Price">
            </div>
            <div class="column-lab">
                <label for="rID">Reprot ID</label>
                <input type="text" id="report" placeholder="ID of the report">
            </div>

            <div class="bottem">
                <input type="button" id="edit_p" value="Edit">
                <input type="button" id="deleteItem_p" value="Delete">
                <input type="button" id="add_p" value="Add">
            </div>
            <form class="file" action="upload.php" method="post" enctype="multipart/form-data">
                <h2>Upload The report</h2>
                <p><input id="ima" type="file" name="image" required></p>
                <p><input id="upl" type="submit" value="Upload"></p>
            </form>
        </div>
    </form>
    <div class="container">
        <h1>Search for Products</h1>
        <form id="search-form">
            <label for="name">Search by name:</label>
            <input type="text" id="name" name="name">
            <label for="type">Search by type:</label>
            <select id="type" name="type">
                <option value="">--Select Type--</option>
                <option value="iPhone">iPhone</option>
                <option value="iPad">iPad</option>
                <option value="Apple Watch">Apple Watch</option>
            </select>
            <label for="rating">Search by rating:</label>
            <select id="rating" name="rating">
                <option value="">--Select Rating--</option>
                <option value="99% New">99% New</option>
                <option value="90% New">90% New</option>
                <option value="85% New">85% New</option>
                <option value="80% New">80% New</option>
            </select>
            <input type="submit" value="Search">
        </form>
```

```
        <br>
        <table>
            <thead>
                <tr>
                    <th>Product/Service Name</th>
                    <th>Description</th>
                    <th>Rating</th>
                    <th>Type</th>
                    <th>Detail</th>
                </tr>
            </thead>
            <tbody id="results-list">
                <!-- Search results will be inserted here -->
            </tbody>
        </table>
    </div>

</div>
```

*Fig 12. Product Management page*

This is the front-end interface of a product management system, including the functions of entering, editing, deleting and querying product information, as well as the display of product listings. Administrator can enter various product information, such as ID, name, description, rating, type, color, display size, storage capacity, Internet version and price, for management purposes. The "Edit" button allows you to modify existing product information, the "Delete" button allows you to delete existing products, and the "Add" button allows you to add new product information. In addition, it also provides the function of uploading reports, and users can upload reports by selecting files.

## Search page

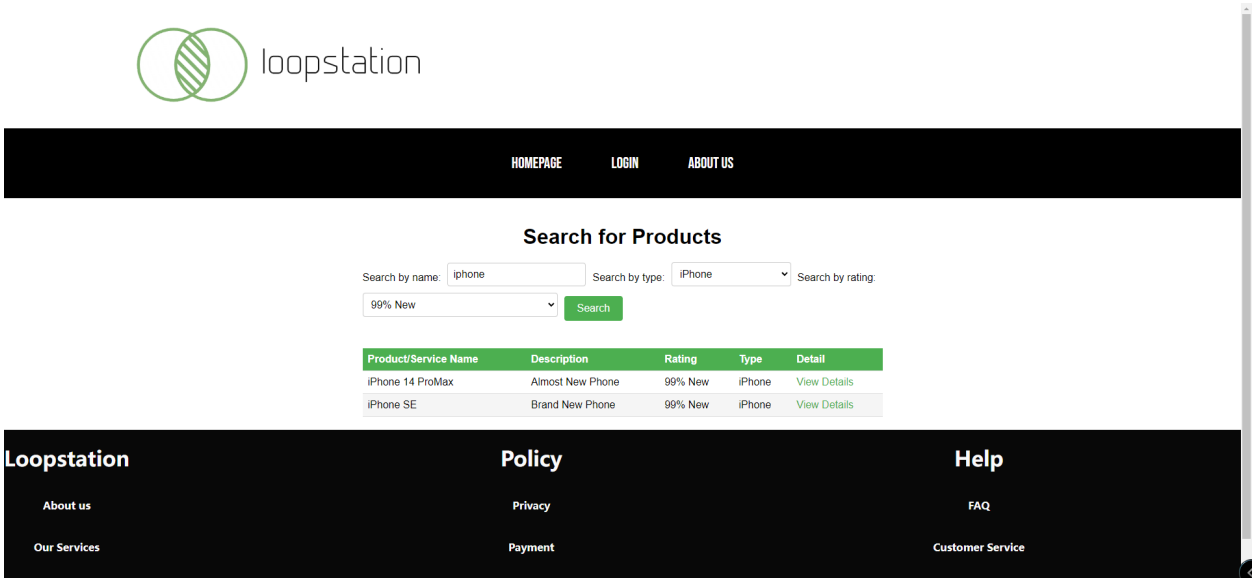



*Fig 13. search page*

```html
<!DOCTYPE html>
<html lang="en">

<head>
    <title>Search Page</title>
    <link rel="stylesheet" href="css/search.css">
</head>

<body>
    <header>
        <img class="logo_homepage" src="image\logo.png">
    </header>
    <nav>
        <a href="homePage"><i class="Homepage"></i> Homepage</a>
        <a href="login"><i class="LoginPage"></i> Login</a>
        <a href="Aboutus"><i class="AboutUsPage"></i> About Us</a>
    </nav>
    <div class="container">
        <h1>Search for Products</h1>
        <form id="search-form">
            <label for="name">Search by name:</label>
            <input type="text" id="name" name="name">
            <label for="type">Search by type:</label>
            <select id="type" name="type">
                <option value="">--Select Type--</option>
                <option value="iPhone">iPhone</option>
                <option value="iPad">iPad</option>
                <option value="Apple Watch">Apple Watch</option>
            </select>
            <label for="rating">Search by rating:</label>
            <select id="rating" name="rating">
                <option value="">--Select Rating--</option>
                <option value="99% New">99% New</option>
                <option value="90% New">90% New</option>
                <option value="85% New">85% New</option>
                <option value="80% New">80% New</option>
            </select>
            <input type="submit" value="Search">
        </form>
```

*Fig 14. search page html code1*

```
        <br>
        <table>
            <thead>
                <tr>
                    <th>Product/Service Name</th>
                    <th>Description</th>
                    <th>Rating</th>
                    <th>Type</th>
                    <th>Detail</th>
                </tr>
            </thead>
            <tbody id="results-list">
                <!-- Search results will be inserted here -->
            </tbody>
        </table>
    </div>
    <footer>
        <div class="column">
            <h1>Loopstation</h1>
            <a href="Aboutus"><i class="AboutUsPage"></i>
                <h4>About us</h4>
            </a>
            <h4>Our Services</h4>
            <h4>History</h4>
        </div>

        <div class="column">
            <h1>Policy</h1>
            <h4>Privacy</h4>
            <h4>Payment</h4>
            <h4>Return</h4>
        </div>
        <div class="column">
            <h1>Help</h1>
            <h4>FAQ</h3>
                <h4>Customer Service</h4>
                <h4>Order</h4>
        </div>
    </footer>
    <script src="/javascript/searchFun.js"></script>
</body>
</html>
```

*Fig 15. search page html code2*

When the user opens this HTML page, they will first see the header and navigation bar of the page, which includes links to the site's logo, home page, login, and about us. Then, the user is presented with the main content of the page, which includes a search form and a form that displays the search results.

In the search form, the user can enter product names, select product types and ratings, and other search criteria. The name is entered by the user input type="text" to get user input, while select id="type" and select id="rating" use the select type to create a drop-down bar for the user to select. The user then clicks the "Search" button to perform the search operation. When the user submits the form, the JavaScript script file searchFun.js captures the form submission event and uses AJAX technology to send a request to the server for information about products that match the search criteria.

When the search results are returned, the JavaScript script will dynamically update the data in the form using DOM operations and present it to the user. In the table, users can view the name, description, rating, type, and details of each product, and learn more about the product through the details link.

In addition, at the bottom of the page, users can see a simple footer with links and information about the site, such as "About Us", "Services", "Privacy Policy ", "Payment Methods", etc. These links provide users with easy access to relevant information and services on the site and improve user experience.

In summary, this HTML page provides a simple and practical user interface for users to quickly find and understand product information by implementing the basic functions of product search and browsing through a clear and simple design and interactive effects.

## Web Services

### How To Make The Communication With Frontend And Backend Services

There are two Web services for our project. One is localhost:3031 for the frontend. Other one is localhost:5000 for the backend. Typically, the frontend is responsible for presenting the user interface and handling user input, while the backend is responsible for processing data and

performing the necessary computations. It's important to ensure that the two services can communicate with each other. The frontend will need to make requests to the backend in order to retrieve data or perform actions, and the backend will need to respond to these requests with the appropriate information.

```
{
  "name": "sec1_gr11_src",
  "version": "1.0.0",
  "description": "ITCS212 - Web Programming",
  "main": "index.js",
  ▷ Debug
  "scripts": {
    "start-frontend": "npm --prefix frontend run start",
    "start-backend": "npm --prefix backend run start",
    "start": "concurrently \"npm run start-frontend\" \"npm run start-backend\""
  },
  "author": "Chusong Xu, Yunduo Luo, Lingrui Dong",
  "license": "ISC",
  "devDependencies": {
    "concurrently": "^8.0.1"
  }
}

```

*Fig 16. The package.json use concurrently package in the outside folder*

In order to achieve the communication with these two Web services, we use concurrently to link the backend and frontend services. Concurrently is a popular package in the Node.js ecosystem that allows you to run multiple commands concurrently. By using concurrently, we can start both servers with a single command and ensure that they run simultaneously. This can make it easier to develop and test your application, since you won't have to manually start each server separately. Concurrently the package will run the frontend and backend services at the same time.

**Backend .env file**

```
backend > .env
BACKEND_PORT=5000
MYSQL_HOST = localhost
MYSQL_USERNAME = loopstation
MYSQL_PASSWORD = 12345678
MYSQL_DATABASE = lsdb
```

*Fig 17. The code of .env file*

## Database

```sql
DROP DATABASE IF EXISTS lsdb;
CREATE DATABASE lsdb;

USE lsdb;

CREATE TABLE IF NOT EXISTS adm_information (
    uid             VARCHAR(50) PRIMARY KEY,
    first_name      VARCHAR(50) NOT NULL,
    last_name       VARCHAR(50) NOT NULL,
    age             INT,
    gender          CHAR(1)      NOT NULL,
    phone           VARCHAR(255),
    email           VARCHAR(255) NOT NULL,
    CONSTRAINT check_uid CHECK (uid REGEXP '^[a][A-Za-z0-9_]*$')
);

CREATE TABLE IF NOT EXISTS adm_login (
    login_id        VARCHAR(50) NOT NULL PRIMARY KEY,
    pass            VARCHAR(50) NOT NULL,
    login_role      ENUM('Boss', 'Product Manager','Tester') NOT NULL,

    CONSTRAINT check_login_id CHECK (login_id REGEXP '^[a][A-Za-z0-9_]*$'),
    CONSTRAINT fk_id  FOREIGN KEY (login_id) REFERENCES adm_information(uid)
);

CREATE TABLE IF NOT EXISTS country (
    c_id        CHAR(2) PRIMARY KEY,
    c_name      VARCHAR(50) NOT NULL
);

CREATE TABLE IF NOT EXISTS address (
    address_id      INT PRIMARY KEY,
    street          VARCHAR(255) NOT NULL,
    city            VARCHAR(255) NOT NULL,
    state           VARCHAR(255) NOT NULL,
    add_country     CHAR(2) NOT NULL,
    zip_code        VARCHAR(10) NOT NULL,
    add_uid         VARCHAR(50) NOT NULL,

    CONSTRAINT fk_add_uid FOREIGN KEY (add_uid) REFERENCES adm_information(uid),
    CONSTRAINT fk_country  FOREIGN KEY (add_country) REFERENCES country(c_id)
);

CREATE TABLE IF NOT EXISTS report(
    rep_id          INT PRIMARY KEY
);
```

```sql
CREATE TABLE IF NOT EXISTS product (
    p_id            INT PRIMARY KEY,
    p_name          VARCHAR(255) NOT NULL,
    p_des           VARCHAR(255) ,
    rating          CHAR(7) NOT NULL,
    p_type          VARCHAR(255) NOT NULL,
    p_color         VARCHAR(255) NOT NULL,
    p_inches        VARCHAR(255) NOT NULL,
    p_memory        VARCHAR(255) NOT NULL,
    p_internet      VARCHAR(255) NOT NULL,
    p_price         DOUBLE NOT NULL,
    p_reportid      INT NOT NULL,

    CONSTRAINT fk_p_repid FOREIGN KEY (p_reportid) REFERENCES report(rep_id)
);
```

```sql
INSERT INTO adm_information (uid, first_name, last_name, age, gender, phone, email) VALUES
("a0001", "John", "Doe", 35, "M", "1234567890", "johndoe@gmail.com"),
("a0002", "Jane", "Doe", 28, "F", "2345678901", "janedoe@outlook.com"),
("a07203", "Robert", "Smith", 42, "M", NULL, "robertsmith@gmail.com"),
("a58204", "Samantha", "Jones", 31, "F", "3456789012", "samanthajones@student.abc.edu"),
("a855", "William", "Lee", 27, "M", "4567890123", "williamlee@student.ac.th"),
("a1236", "Emily", "Brown", 24, "F", "09514269", "emilybrown@outlook.com"),
("a14587", "David", "Johnson", 38, "M", NULL, "davidjohnson@gmail.com"),
("a741569", "Melissa", "Davis", 29, "F", "8901234567", "melissadavis@mail.com");

INSERT INTO adm_login (login_id, pass, login_role) VALUES
('a0001', '123456', 'Boss'),
('a0002', 'sdsd1512', 'Product Manager'),
('a07203', 'Ssdaa132', 'Tester'),
('a58204', 'asdjhggh', 'Product Manager'),
('a855', 'hgjdf82gh6', 'Tester'),
('a1236', 'ghdfghb442', 'Tester'),
('a14587', '123abc', 'Tester'),
('a741569', '20000920abc', 'Tester');
```

```sql
INSERT INTO country (c_id,c_name) VALUES
('AF', 'Afghanistan'),
('AX', 'Aland Islands'),
('AL', 'Albania'),
('DZ', 'Algeria'),
('AS', 'American Samoa'),
('AD', 'Andorra'),
('AO', 'Angola'),
('AI', 'Anguilla'),
('AQ', 'Antarctica'),
('AG', 'Antigua and Barbuda'),
('AR', 'Argentina'),
('AM', 'Armenia'),
('AW', 'Aruba'),
('AU', 'Australia'),
('AT', 'Austria'),
('AZ', 'Azerbaijan'),
('BS', 'Bahamas'),
('BH', 'Bahrain'),
('BD', 'Bangladesh'),
('BB', 'Barbados'),
('BY', 'Belarus'),
('BE', 'Belgium'),
('BZ', 'Belize'),
('BJ', 'Benin'),
('BM', 'Bermuda'),
('BT', 'Bhutan'),
('BO', 'Bolivia'),
('BQ', 'Bonaire, Sint Eustatius and Saba'),
('BA', 'Bosnia and Herzegovina'),
('BW', 'Botswana'),
('BV', 'Bouvet Island'),
('BR', 'Brazil'),
('IO', 'British Indian Ocean Territory'),
('BN', 'Brunei Darussalam'),
('BG', 'Bulgaria'),
('BF', 'Burkina Faso'),
('BI', 'Burundi'),
('KH', 'Cambodia'),
('CM', 'Cameroon'),
('CA', 'Canada'),
('CV', 'Cape Verde'),
('KY', 'Cayman Islands'),
('CF', 'Central African Republic'),
('TD', 'Chad'),
('CL', 'Chile'),
('CN', 'China'),
('CX', 'Christmas Island'),
('CC', 'Cocos (Keeling) Islands'),
```

```
INSERT INTO address (address_id, street, city, state, add_country, zip_code, add_uid) VALUES
(1, '123 Main St', 'Mar Lee', 'Colorado', 'US', '12345', 'a0001'),
(2, '456 Oak Ave', 'Taipei', 'Taipei', 'TW', '23456', 'a0002'),
(3, '789 Central Park Ln', 'New York', 'New York', 'US', '34567', 'a07203'),
(4, 'Soi Samsen 6', 'Bangkok', 'Bangkok', 'TH', '45678', 'a58204'),
(5, '555 Elm St', 'Salaya', 'Nakhon Pathom', 'TH', '56789', 'a855'),
(6, 'Cuihu St', 'Kunming', 'Yunnan', 'CN', '67890', 'a1236'),
(7, '777 Broadway', 'Manchester', 'Manchester', 'UK', '78901', 'a14587'),
(8, '999 Church St', 'Albuquerque', 'New Mexico', 'US', '89012', 'a741569');

INSERT INTO report (rep_id) VALUES
(1),
(2),
(3),
(4),
(5),
(6),
(7),
(8),
(9),
(10);

-- In p_id the first number mean the product type
-- Phone: 1
-- Pad: 2
-- Wearable: 3
INSERT INTO product (p_id, p_name, p_des, rating, p_type,p_color, p_inches, p_memory, p_internet, p_price, p_reportid) VALUES
(100001,"iPhone 14 ProMax","Almost New Phone","99% New","iPhone","Night Purple","6.69 inches","256G","5G","31000.00",1),
(100002,"iPhone 13","Small Scratches","80% New","iPhone","Gold","6.06 inches","64G","5G","12000.00",2),
(100003,"iPhone 13 Pro","Cost-effective Choice","85% New","iPhone","Black","6.06 inches","128G","5G","19000.00",3),
(100004, "iPhone SE", "Brand New Phone", "99% New", "iPhone","White","4.7 inches","64G","4G","5800.00", 4),
(100005, "iPhone XR", "Good Condition Phone", "90% New", "iPhone","White","6.06 inches","64G","4G","7490.00", 5),

(200001,"iPad 7","Small Scratches","90% New ","iPad","Space Gray","10.2 inches","64G","WiFi","6900.00",6),
(200002, "iPad Air", "Slight Wear and Tear", "85% New", "iPad","Black","10.86 inches","64G","4G","2700.00",7),
(200003, "iPad Pro 12.9-inch", "Like New iPad", "98% New", "iPad","Black","12.9 inches","256G","WiFi","23500.00",8),

(300001,"Apple Watch SE","Almost New Watch","99% New","Apple Watch","White","44 mm","32G","GPS","2900.00",9),
(300002, "Apple Watch Series 7", "Like New Watch", "98% New", "Apple Watch","White","45 mm","32G","GPS","7800.00",10);
```

*Fig 18. Database sql code*

We set up a database named 'lsdb', which contains five tables: adm_information, adm_login, country, address, and product. The adm_information table stores user information such as first and last name, age, gender, phone, and email, with a constraint on the uid format. The adm_login table links to the adm_information table and stores the login information, including the role of the user (Boss, Product Manager, or Tester). The country table stores a list of country codes and their corresponding names. The address table links to both the adm_information and country tables and contains address information, including street, city, state, country, and zip code. The product table links to a report table and stores product-related information, such as name, description, rating, type, color, size, memory, internet, price, and a report ID. The code also contains sample data inserts for each table.

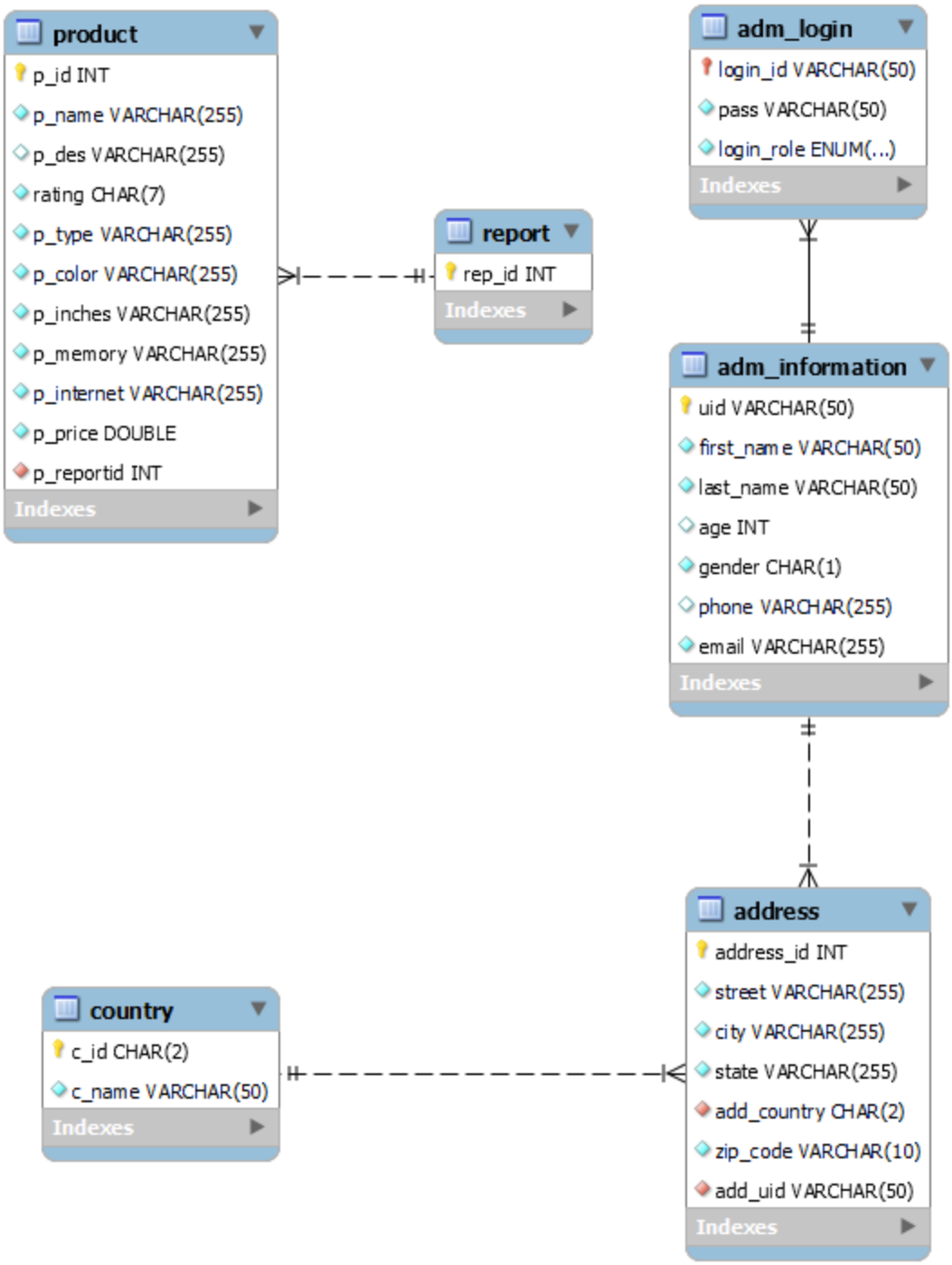



*Fig 19. Database EER diagram*

**Connect To The Database**

```
const express = require("express");
const sqlite3 = require('sqlite3').verbose();
const cors = require('cors');
const app = express();
const router = express.Router();
const bp = require("body-parser");
const dotenv = require("dotenv");

const port = process.env.BACKEND_PORT || 5000;



app.use("/", router);
var jsonParser = bp.json();
dotenv.config();
router.use(cors());
app.use(cors());
app.use(express.json());
app.use(cors({ origin: '*' }));

//connect to database
const mysql = require("mysql2");
var connection = mysql.createConnection({
  host: process.env.MYSQL_HOST,
  user: process.env.MYSQL_USERNAME,
  password: process.env.MYSQL_PASSWORD,
  database: process.env.MYSQL_DATABASE
});

/*Connect to DB*/
connection.connect(function (err) {
  if (err) throw err;
  console.log(`Connected DB: ${process.env.MYSQL_DATABASE}`);
});
```

*Fig 20. The index.js in backend folder*

## The Frontend Javascript

```
// Import required modules
const express = require('express');
const port = 3031;
const app = express();
const router = express.Router();
const dotenv = require('dotenv');
const cors = require("cors");
const path = require('path')
const morgan = require('morgan')
// Load environment variables from .env file
dotenv.config();
// Set up view engine and views folder
app.set('view engine', 'ejs');
app.set('views', './views');
// Middleware for serving static files, JSON, URL encoding, and logging
app.use(express.static('public'))
app.use(express.json())
app.use(morgan('dev'))
app.use(express.urlencoded({ extended: false }))
app.use(cors());
// Initialize and use cookie-parser middleware
const cookieParser = require('cookie-parser');
app.use(cookieParser());
app.use(cors({ origin: '*' }));
// Use the router middleware
app.use(router);
// Define routes for serving pages from the public folder
app.get('/', (req, res) => {
    res.status(200).sendFile(path.resolve(__dirname, 'public', 'homePage.html'));
});
app.get('/login', (req, res) => {
    res.status(200).sendFile(path.resolve(__dirname, 'public', 'login.html'));
});
app.get('/homePage', (req, res) => {
    res.status(200).sendFile(path.resolve(__dirname, 'public', 'homePage.html'));
});
app.get('/search', (req, res) => {
    res.status(200).sendFile(path.resolve(__dirname, 'public', 'search.html'));
});
app.get('/Aboutus', (req, res) => {
    res.status(200).sendFile(path.resolve(__dirname, 'public', 'Aboutus.html'));
});
app.get('/productManage', (req, res) => {
    res.status(200).sendFile(path.resolve(__dirname, 'public', 'productManage.html'));
});
app.get('/userManagement', (req, res) => {
    res.status(200).sendFile(path.resolve(__dirname, 'public', 'userManagement.html'));
});
app.get('/100001', (req, res) => {
    res.status(200).sendFile(path.resolve(__dirname, 'public', '100001.html'));
});
app.get('/100002', (req, res) => {
    res.status(200).sendFile(path.resolve(__dirname, 'public', '100002.html'));
});
app.get('/100003', (req, res) => {
    res.status(200).sendFile(path.resolve(__dirname, 'public', '100003.html'));
});
app.get('/200001', (req, res) => {
    res.status(200).sendFile(path.resolve(__dirname, 'public', '200001.html'));
});
app.get('/300001', (req, res) => {
    res.status(200).sendFile(path.resolve(__dirname, 'public', '300001.html'));
});

app.listen(port, () => {
    console.log(`Server listening on port: ${port}`)
```

*Fig 21. The index.js file for frontend*

This index.js code sets up an Express server that listens for HTTP requests on port 3031. It also uses middleware like cors to handle cross-origin resource sharing, morgan to log HTTP requests, and cookie-parser to handle cookies.

It then defines routes for serving static HTML files located in the public folder when certain URL paths are requested. For example, when a GET request is made to the path '/login', it responds by sending the login.html file from the public folder. The same is done for other routes such as '/homePage', '/search', '/Aboutus', '/productManage', and '/userManagement'.

Finally, the server is started using the app.listen() method, which logs a message to the console indicating that the server is listening on the specified port. Overall, this code is responsible for setting up a basic Express server that serves static HTML pages for a frontend application.

## Product Search

```
function displayResults(results) {
    const resultsList = document.getElementById('results-list');
    resultsList.innerHTML = '';

    if (results.length === 0) {
        const noResults = document.createElement('tr');
        const noResultsCell = document.createElement('td');
        noResultsCell.textContent = 'No results found';
        noResultsCell.setAttribute('colspan', '5');
        noResults.appendChild(noResultsCell);
        resultsList.appendChild(noResults);
    } else {
        for (const result of results) {
            const row = document.createElement('tr');

            const nameCell = document.createElement('td');
            nameCell.textContent = result.p_name;
            row.appendChild(nameCell);

            const descriptionCell = document.createElement('td');
            descriptionCell.textContent = result.p_des;
            row.appendChild(descriptionCell);

            const ratingCell = document.createElement('td');
            ratingCell.textContent = result.rating;
            row.appendChild(ratingCell);

            const typeCell = document.createElement('td');
            typeCell.textContent = result.p_type;
            row.appendChild(typeCell);

            const detailCell = document.createElement('td');
            const detailLink = document.createElement('a');
            detailLink.href = `./${result.p_id}`; //  have a details page with the product id in the URL
            detailLink.textContent = 'View Details';
            detailCell.appendChild(detailLink);
            row.appendChild(detailCell);

            resultsList.appendChild(row);
        }
    }
}

document.getElementById('search-form').addEventListener('submit', async (event) => {
    event.preventDefault();
    const name = document.getElementById('name').value;
    const type = document.getElementById('type').value;
    const rating = document.getElementById('rating').value;

    try {
        const response = await fetch('http://localhost:5000/search', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
            },
            body: JSON.stringify({ name, type, rating }),
        });

        if (response.ok) {
            const results = await response.json();
            displayResults(results);
        } else {
            console.error('Error:', response.statusText);
        }
    } catch (error) {
        console.error('Error:', error);
    }
});
```

*Fig 22. The searchFun.js file for search page*

```
app.post('/search', jsonParser, async (req, res) => {
  const { name, type, rating } = req.body;
  let query = "SELECT * FROM product ";
  const conditions = [];
  //use name to search
  if (name) {
    conditions.push(`p_name LIKE '%${name}%'`);
  }
  //use type to search
  if (type) {
    conditions.push(`p_type='${type}'`);
  }
  //use rating to search
  if (rating) {
    conditions.push(`rating = '${rating}'`);
  }
  //use name and type to search
  if (name && type) {
    conditions.push(`p_name LIKE '%${name}%' AND p_type='${type}'`);
  }
  //use name and rating to search
  if (name && rating) {
    conditions.push(`p_name LIKE '%${name}%' AND rating = '${rating}'`);
  }
  //use type and rating to search
  if (type && rating) {
    conditions.push(`p_type='${type}' AND rating = '${rating}'`);
  }
  //use name, type and rating to search
  if (name && type && rating) {
    conditions.push(`p_name LIKE '%${name}%' AND p_type='${type}' AND rating = '${rating}'`);
  }
  //add conditions to query
  if (conditions.length > 0) {
    query += "WHERE " + conditions.join(" AND ");
  }
  try {
    connection.query(query, (err, results, fields) => {
      if (err) {
        console.error(err);
        res.status(500).json({ error: 'Internal Server Error' });
      } else {
        res.status(200).json(results);
      }
    });
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: 'Internal Server Error' });
  }
});
```

*Fig 23. The index.js file from the backend server related to the search function*

We implemented the search functionality for products on a website using a combination of HTML, JavaScript, and a backend server. The main components of this implementation include a search form in the 'search.html' file, JavaScript code in the 'searchFun.js' file, and server-side code in the 'index.js' file. In the 'search.html' file, we created a search form where users can input product name, select product type, and choose a product rating to filter their search. Once the form is submitted, the search criteria are sent to the 'searchFun.js' file, which handles the form submission event. In the 'searchFun.js' file, we used the Fetch API to send a POST request to the backend server with the search criteria as JSON data. Upon receiving a response from the server, the search results are displayed in a table format using the 'displayResults' function. On the backend side, in the 'index.js' file, we set up an endpoint to handle the POST request sent by the front-end. Using the search criteria received in the request body, we constructed a SQL query to fetch relevant data from the product table. We used a series of conditional statements to build the query dynamically based on the user's input. Once the query is executed and the results are retrieved, they are sent back to the front-end as a JSON response. If an error occurs during the process, the server responds with a 500 status code and an error message. This implementation allows users to search for products on the website based on their preferences and displays the results in a clear and organized manner.

**Display Product Detail**

```javascript
function updateProductDetails(product) {
    // Update the title and product details
    document.querySelector('.Products_title').textContent = product.p_name;
    document.getElementById('pd_rating').textContent = product.p_rate;

    // Update the left image and information

    document.getElementById('pd_name').textContent = product.p_name;
    document.getElementById('pd_inches').textContent = product.p_inches;
    document.getElementById('pd_memory').textContent = product.p_memory;
    document.getElementById('pd_color').textContent = product.p_color;
    document.getElementById('pd_internet').textContent = product.p_internet;

    // Update the price
    document.getElementById('pd_price').dataset.price = product.p_price;
    document.getElementById('pd_price').textContent = '฿' + product.p_price;


}

// Get productId from URL
function getProductIdFromUrl() {
    const path = window.location.pathname;
    const filename = path.split('/').pop();
    const productId = filename.split('.')[0];
    return productId;
}
const productId = getProductIdFromUrl();
fetchProductDetails(productId);


async function fetchProductDetails(productId) {
    try {
        const response = await fetch(`http://localhost:5000/api/productdetail/${productId}`);

        if (response.status === 404) {
            console.error('Product not found');
            return;
        }
        const product = await response.json();
        updateProductDetails(product.data);
    } catch (error) {
        console.error('Error fetching product details:', error);
    }
}

```

*Fig 24. The productDetail.js*

```
//This is for product detail page
app.get('/api/productdetail/:id', function (req, res) {
  let productId = req.params.id;
  if (!productId) {
    return res.status(400).send({ error: true, message: 'Please provide product_id' });
  }

  connection.query('SELECT * FROM product WHERE p_id = ?', [productId], function (error, results) {
    if (error) throw error;
    if (results.length === 0) {
      return res.status(404).send({ error: true, message: 'Product not found' });
    }
    return res.send({ error: false, data: results[0], message: 'Product details fetched successfully.' });
  });
});
```

*Fig 25. The index.js file from the backend server related to the show product detail*

We implemented the product detail functionality for a website using a combination of HTML, JavaScript, and a backend server. The main components of this implementation include the product detail page in the ‘100001.html’, ‘100002.html’, ‘100003.html’, ‘200001.html’, ‘300001.html’ file, JavaScript code in the 'productDetail.js' file, and server-side code in the 'index.js' file. In the ‘100001.html’, ‘100002.html’, ‘100003.html’, ‘200001.html’, ‘300001.html’ file,we created a layout for displaying product details, including images, product title, description, price, and a buy button. There is also a currency selection dropdown to allow users to view the price in their preferred currency. In the 'productDetail.js' file, We first extracted the product ID from the URL using the 'getProductIdFromUrl' function. Then, I used the Fetch API to send a GET request to the backend server with the product ID as a URL parameter. Upon receiving a response from the server, we updated the product details on the page using the 'updateProductDetails' function. On the backend side, in the 'index.js' file, we set up an endpoint to handle the GET request sent by the front-end. The product ID is extracted from the URL parameter, and a SQL query is executed to fetch the corresponding product details from the product table. If the product is found, the server responds with a JSON object containing the product details. If not, the server responds with a 404 status code and an error message. This implementation allows users to view detailed information about a specific product on the website, providing them with all the necessary information before making a purchase.

## Product Management

```
function displayResults(results) {
    const resultsList = document.getElementById('results-list');
    resultsList.innerHTML = '';

    if (results.length === 0) {
        const noResults = document.createElement('tr');
        const noResultsCell = document.createElement('td');
        noResultsCell.textContent = 'No results found';
        noResultsCell.setAttribute('colspan', '5');
        noResults.appendChild(noResultsCell);
        resultsList.appendChild(noResults);
    } else {
        for (const result of results) {
            const row = document.createElement('tr');

            const nameCell = document.createElement('td');
            nameCell.textContent = result.p_name;
            row.appendChild(nameCell);

            const descriptionCell = document.createElement('td');
            descriptionCell.textContent = result.p_des;
            row.appendChild(descriptionCell);

            const ratingCell = document.createElement('td');
            ratingCell.textContent = result.rating;
            row.appendChild(ratingCell);

            const typeCell = document.createElement('td');
            typeCell.textContent = result.p_type;
            row.appendChild(typeCell);

            const detailCell = document.createElement('td');
            const detailLink = document.createElement('a');
            detailLink.href = `./${result.p_id}.html`; // Assuming you have a details page with the product id in the URL
            detailLink.textContent = 'View Details';
            detailCell.appendChild(detailLink);
            row.appendChild(detailCell);

            resultsList.appendChild(row);
        }
    }
}

document.getElementById('search-form').addEventListener('submit', async (event) => {
    event.preventDefault();
    const name = document.getElementById('name').value;
    const type = document.getElementById('type').value;
    const rating = document.getElementById('rating').value;

    try {
        const response = await fetch('http://localhost:5000/search', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
            },
            body: JSON.stringify({ name, type, rating }),
        });

        if (response.ok) {
            const results = await response.json();
            displayResults(results);
        } else {
            console.error('Error:', response.statusText);
        }
    } catch (error) {
        console.error('Error:', error);
    }
});
```

```
document.getElementById("add_p").addEventListener("click", async () => {

    const product = {
        p_id: document.getElementById("productID").value,
        p_name: document.getElementById("productName").value,
        p_des: document.getElementById("des").value,
        rating: document.getElementById("rate").value,
        p_type: document.getElementById("ptype").value,
        p_color: document.getElementById("color").value,
        p_inches: document.getElementById("inche").value,
        p_memory: document.getElementById("memory").value,
        p_internet: document.getElementById("internet").value,
        p_price: document.getElementById("price").value,
        p_reportid: document.getElementById("report").value,
      };
      console.log("Sending product data:", JSON.stringify(product, null, 2));
    try {
        const response = await fetch(`http://localhost:5000/api/product`, {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify(product),
      });

      if (response.ok) {
        alert("Product added successfully");
        location.reload();
      } else {
        alert("Error adding product");
      }
    } catch (error) {
      console.error("Error:", error);
      alert("Error adding product");
    }
  });


  document.getElementById('edit_p').addEventListener('click', async () => {
    const productId = document.getElementById('productID').value;
    const product = {
      p_name: document.getElementById('productName').value,
      p_des: document.getElementById('des').value,
      rating: document.getElementById('rate').value,
      p_type: document.getElementById('ptype').value,
      p_color: document.getElementById('color').value,
      p_inches: document.getElementById('inche').value,
      p_memory: document.getElementById('memory').value,
      p_internet: document.getElementById('internet').value,
      p_price: document.getElementById('price').value,
      p_reportid: document.getElementById('report').value,
    };
```

```
  try {
    const response = await fetch(`http://localhost:5000/api/product/${productId}`, {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(product),
    });

    if (response.ok) {
      alert('Product updated successfully');
      location.reload();
    } else {
      alert('Error updating product');
    }
  } catch (error) {
    console.error('Error:', error);
    alert('Error updating product');
  }
});

document.getElementById("deleteItem_p").addEventListener("click", async () => {
  const productId = document.getElementById("productID").value;

  try {
    const response = await fetch(`http://localhost:5000/api/product/${productId}`, {
      method: "DELETE",
    });

    if (response.ok) {
      alert("Product deleted successfully");
      location.reload();
    } else {
      alert("Error deleting product");
    }
  } catch (error) {
    console.error("Error:", error);
    alert("Error deleting product");
  }
});
```

*Fig 26. The productManage.js*

**Insert Product**

```
app.post('/api/product', function (req, res) {
  let product = req.body;
  if (!product) {
    return res.status(400).send({ error: true, message: 'Please provide complete product information' });
  }

  // Insert report
  connection.query("INSERT INTO report (rep_id) VALUES (?)", product.p_reportid, function (error, reportResults) {
    if (error) {
      console.error("Error inserting report:", error);
      throw error;
    }
    // Insert product
    connection.query("INSERT INTO product SET ? ", product, function (error, productResults) {
      if (error) throw error;
      return res.send({ error: false, data: productResults.insertId, message: 'New product has been created successfully.' });
    });
  });
});
```

*Fig 27. The index.js code for insert product*

In this implementation, we have created a webpage for managing products, where users can add, edit, delete, and search for products. The main focus here is on the "Insert product" functionality. To accomplish this, we have used JavaScript for handling the front-end logic and communicating with the back-end server. Specifically, an event listener is attached to the "Add" button (with the ID "add_p"). When the button is clicked, the event listener collects the product information from the input fields and creates a product object. This object is then sent as a JSON payload to the back-end server using the fetch function with a POST request. The back-end server, which is built using Node.js and Express, defines an API endpoint /api/product for handling the POST request. Upon receiving the request, the server first checks if the product object is complete. If the object is valid, it proceeds to insert a new record into the report table, followed by the product table in the database. If the insertion is successful, the server sends back a response indicating that the new product has been created successfully.

**Update Product**

```
app.put('/api/product/:id', function (req, res) {
  let productId = req.params.id;
  let product = req.body;
  if (!productId) {
    return res.status(400).send({ error: true, message: 'Please provide product_id' });
  }
  if (!product) {
    return res.status(400).send({ error: true, message: 'Please provide product information' });
  }
  connection.query('UPDATE product SET ? WHERE p_id = ?', [product, productId], function (error, results) {
    if (error) throw error;

    return res.send({ error: false, data: results.affectedRows, message: 'Product has been updated successfully.' });
  });
});
```

*Fig 28. The index.js code for update product*

In order to implement the "update product" functionality, I have designed a web page called "productManage.html" that includes a form for editing product details. The JavaScript file, "productManage.js", contains event listeners and functions to handle the update process. When the "edit_p" button is clicked, an event listener is triggered, which retrieves the updated product information from the input fields on the form. The updated product data is sent to the backend server using a PUT request to the "/api/product/:id" endpoint. The server-side code, written in Node.js, defines the endpoint and processes the request. The server extracts the product ID from the request parameters and the updated product information from the request body. It then runs an SQL query to update the corresponding product record in the database. If the update is successful, the server sends a response with a success message, and the frontend displays an alert informing the user that the product has been updated successfully. If there's an error, an error message is displayed to the user instead. This implementation allows users to easily update product information on the website.

**Delete Product**

```
app.delete('/api/product/:id', function (req, res) {
  let productId = req.params.id;
  if (!productId) {
    return res.status(400).send({ error: true, message: 'Please provide product_id' });
  }

  // Get p_reportid for the product
  connection.query('SELECT p_reportid FROM product WHERE p_id = ?', [productId], function (error, reportIdResults) {
    if (error) throw error;
    if (reportIdResults.length === 0) {
      return res.status(404).send({ error: true, message: 'Product not found' });
    }
    const reportId = reportIdResults[0].p_reportid;

    // Delete product
    connection.query('DELETE FROM product WHERE p_id = ?', [productId], function (error, productResults) {
      if (error) throw error;

      // Delete report
      connection.query('DELETE FROM report WHERE rep_id = ?', [reportId], function (error, reportResults) {
        if (error) throw error;

        // Send response
        return res.send({
          error: false,
          data: {
            productRowsAffected: productResults.affectedRows,
            reportRowsAffected: reportResults.affectedRows,
          },
          message: 'Product and associated report have been deleted successfully.',
        });
      });
    });
  });
});
```

*Fig 29. The index.js code for delete product*

In this implementation, the deletion of a product is achieved using a combination of frontend and backend code. On the frontend, an event listener is added to the "deleteItem_p" button, which listens for a click event. When the button is clicked, the function asynchronously sends an HTTP DELETE request to the backend with the product ID as a parameter in the URL. On the backend, an Express app is set up to handle the DELETE request at the specified route '/api/product/:id', where ':id' represents the product ID. First, the product ID is retrieved from the request parameters, and a check is performed to ensure the product ID exists. Then, the database is queried to get the report ID associated with the product. If the product is found, it proceeds to delete the product and the associated report from their respective tables using separate SQL queries. Finally, a response is sent to the frontend, indicating the number of rows affected in both the product and report tables, along with a success message.

## Login Javascript

```
document.addEventListener("DOMContentLoaded", () => {
    const loginForm = document.getElementById("loginForm");

    loginForm.addEventListener("submit", async (event) => {
        event.preventDefault();

        const loginId = document.getElementById("log_id").value;
        const password = document.getElementById("password").value;

        try {
            const response = await fetch("http://localhost:5000/login", {
                method: "POST",
                headers: {
                    "Content-Type": "application/json",
                },
                body: JSON.stringify({ loginId, password }),
            });

            if (response.ok) {
                const data = await response.json();
                alert("Login successful!");

                const loginRole = data.data.login_role;
                if (["Tester", "Boss", "Product Manager"].includes(loginRole)) {
                    window.location.href = "userManagement";
                }
            } else {
                alert("Invalid login credentials");
            }
        } catch (error) {
            console.error("Error:", error);
            alert("Error during login");
        }
    });
});
```

*Fig 30. The Login.js file for Login*

This code is a Web API-based user login function that uses the Fetch API to send requests to the server and receive responses. When the user enters the login ID and password in

the login form and submits the form, JavaScript blocks the default form behavior and sends the login information to the server in JSON format using the POST method. If the server returns an OK response, JavaScript displays a "login successful" message on the front end and then checks if the login role is "Tester", "Boss " or "Product Manager". If so, JavaScript will redirect the page to the user management page; otherwise, the front-end will prompt "Invalid login credentials".

## AdmiFun.js

```
function displayResults(results) {
    const resultsList = document.getElementById('admi-list');
    resultsList.innerHTML = '';

    if (results.length === 0) {
        const noResults = document.createElement('tr');
        const noResultsCell = document.createElement('td');
        noResultsCell.textContent = 'No results found';
        noResultsCell.setAttribute('colspan', '5');
        noResults.appendChild(noResultsCell);
        resultsList.appendChild(noResults);
    } else {
        for (const result of results) {
            const row = document.createElement('tr');

            const idCell = document.createElement('td');
            idCell.textContent = result.uid;
            row.appendChild(idCell);

            const FirstnameCell = document.createElement('td');
            FirstnameCell.textContent = result.first_name;
            row.appendChild(FirstnameCell);

            const LastnameCell = document.createElement('td');
            LastnameCell.textContent = result.last_name;
            row.appendChild(LastnameCell);

            const ageCell = document.createElement('td');
            ageCell.textContent = result.age;
            row.appendChild(ageCell);

            const genderCell = document.createElement('td');
            genderCell.textContent = result.gender;
            row.appendChild(genderCell);


            const phoneCell = document.createElement('td');
            phoneCell.textContent = result.phone;
            row.appendChild(phoneCell);

            const eamilCell = document.createElement('td');
            eamilCell.textContent = result.email;
            row.appendChild(eamilCell);
            resultsList.appendChild(row);
        }
    }
```

```
document.getElementById('searchUsers').addEventListener('click', async (event) => {
    event.preventDefault();
    const id = document.getElementById('searchid').value;
    const fname = document.getElementById('search_fname').value;
    const lname = document.getElementById('search_lname').value;
    const eamil = document.getElementById('search_email').value;
    const gender = document.getElementById('searchgender').value;

    try {
        const response = await fetch('http://localhost:5000/searchAdmi', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
            },
            body: JSON.stringify({ condition: { uid: id, first_name: fname, last_name: lname, email: eamil, gender: gender }

            }),

        });
        console.log('resbosd')
        if (response.ok) {
            const results = await response.json();
            console.log('Response data:', results); // Add this lineto find the bug
            displayResults(results);
        } else {
            console.error('Error:', response.statusText);
        }
    } catch (error) {
        console.error('Error:', error);
    }
});
```

```
//view all
document.getElementById('viewall_u').addEventListener('click', async () => {
    try {
        const response = await fetch('http://localhost:5000/admis', {
            method: 'GET',
            headers: {
                'Content-Type': 'application/json',
            },
        })

        if (response.ok) {
            const results = await response.json();
            displayResults(results.data);
        } else {
            console.error('Error:', response.statusText);
        }
    } catch (error) {
        console.error('Error:', error);
    }
});
```

```
//Add admi
document.getElementById("add_u").addEventListener("click", async () => {
    const id = document.getElementById('userID').value;
    const fname = document.getElementById('userFname').value;
    const lname = document.getElementById('userLname').value;
    const age = document.getElementById('Age').value;
    const gender = document.getElementById('Gender').value;
    const phone = document.getElementById('Phone').value;
    const email = document.getElementById('Email').value;
    const loginId = document.getElementById('loginId').value;
    const password = document.getElementById('password').value;
    const loginRole = document.getElementById('loginRole').value;

    try {
        const response = await fetch(`http://localhost:5000/admi`, {
            method: "POST",
            headers: {
                "Content-Type": "application/json",
            },
            body: JSON.stringify({
                Adminfo: { uid: id, first_name: fname, last_name: lname, age: age, gender: gender, phone: phone, email: email },
                Loginfo: { login_id: loginId, pass: password, login_role: loginRole }
            }),
        });

        if (response.ok) {
            alert("Administrator added successfully");
            location.reload();
        } else {
            alert("Error adding administrator");
        }
    } catch (error) {
        console.error("Error:", error);
        alert("Error adding administrator");
    }
});
```

```
document.getElementById("deleteItem_u").addEventListener("click", async () => {
    const admiId = document.getElementById("userID").value;

    try {
        const response = await fetch(`http://localhost:5000/admi/${admiId}`, {
            method: "DELETE",
        });

        if (response.ok) {
            alert("Admi deleted successfully");
            location.reload();
        } else {
            alert("Error deleting Admi");
        }
    } catch (error) {
        console.error("Error:", error);
        alert("Error deleting Admi");
    }
});



document.getElementById('viewall_u').addEventListener('click', async () => {
    try {
        const response = await fetch('http://localhost:5000/admis');

        if (response.ok) {
            const results = await response.json();
            displayResults(results.data);
        } else {
            console.error('Error:', response.statusText);
        }
    } catch (error) {
        console.error('Error:', error);
    }
});
```

*Fig 31. AdmiFun.js*

This code is a JavaScript script for managing administrator information. The main functions include the following:

1. displayResults: the displayResults function is used to display the searched administrator information on the page.
2. search for administrators: listen to the 'searchUsers' button click event and send a POST request to the backend to search for administrators based on the input criteria.
3. view all administrators: listen to the 'viewall_u' button click event, send a GET request to the backend to get all the administrator information, and call the displayResults function to display the results.

4. add administrator: listen to the 'add_u' button click event, collect input data from the form, and send a POST request to the backend to add a new administrator. 5. edit administrator: listen to the 'add_u' button click event, and send a POST request to the backend to add a new administrator.

5. edit administrator: listen to the 'edit_u' button click event, collect input data from the form, and send a PUT request to the backend to update the information of the specified administrator.

6. delete administrator: listen to the 'deleteItem_u' button click event and send a DELETE request to the backend to delete the specified administrator based on the user ID.

7. view all administrators again: listen to the 'viewall_u' button click event, send a GET request to the backend to get all administrators' information, and call the displayResults function to display the result.

```
    <!-- Search result table -->
    <table>
        <thead>
            <tr>
                <th>User ID</th>
                <th>First Name</th>
                <th>Last Name</th>
                <th>Gender</th>
                <th>Age</th>
                <th>Phone</th>
                <th>Email</th>
            </tr>
        </thead>
        <tbody id="admi-list">
            <!-- Search result will be added here using JavaScript -->
        </tbody>
    </table>
    </table>
</div>
```

*Fig 32. Search result List*

And it will play result at this place.

## Index.js

### Search Function

```
app.post('/searchAdmi', jsonParser, function (req, res) {

  console.log(req.body.condition);

  let adm_id = (req.body.condition.uid ? " AND (uid = '" + req.body.condition.uid + "')" : "");
  let adm_fname = "%" + (req.body.condition.first_name || "").toLowerCase() + "%"; // ignore the case
  let adm_lname = "%" + (req.body.condition.last_name || "").toLowerCase() + "%";
  let adm_age = (req.body.condition.age ? " AND (age = " + parseInt(req.body.condition.age) + ")" : "");
  let adm_gen = (req.body.condition.gender ? " AND (gender = '" + req.body.condition.gender + "')" : "");
  let adm_phone = (req.body.condition.phone ? " AND (phone = '" + req.body.condition.phone + "')" : "");
  let adm_email = (req.body.condition.email ? " AND (email = '" + req.body.condition.email + "')" : "");

  let qur = "SELECT uid, first_name, last_name, age, gender, phone, email"
    + " FROM adm_information"
    + " WHERE (LOWER(first_name) LIKE ?)"
    + " AND (LOWER(last_name) LIKE ?)"
    + adm_id + adm_age + adm_gen + adm_phone + adm_email // Add additional filters
    + " ORDER BY uid ASC";
  connection.query(qur, [adm_fname, adm_lname], function (error, results) {
    if (error) throw error;
    console.log('Query results:', results);
    // return res.send({
    //   // error: false,
    //   // data: results,
    //   // message: 'check successfully'
    // });
    return res.status(200).json(results)
  });
});
```

*Fig 33. Search Function*

This code is a back-end API for searching administrator information. a POST route named /searchAdmi is defined to handle search requests sent from the front-end. When a request is received, the server extracts the query criteria from the body of the request and then constructs a SQL query statement based on those criteria.

Parse the JSON data in the request body using jsonParser. Extract search criteria from the request body, such as admin ID, first name, last name, age, gender, phone and email. Construct SQL query statements based on the extracted conditions. Use the LOWER() function to ignore case and use the LIKE operator for fuzzy matching. Other conditions are matched exactly using the EQUAL operator. Send a query to the database and pass the query result to the callback function.

**View all function**

```
// view all Admi
// Test the select all admi, using GET url: http://localhost:5000/admis
app.get("/admis", jsonParser, function (req, res) {
  connection.query('SELECT * FROM adm_information Order by uid ASC', function (error, results) {
    if (error) throw error;
    return res.send({
      error: false,
      data: results,
      message: 'Admi List'
    })
  })
})
```

*Fig 34. VIEW ALL  Function*

This show all administrator to the user.

**Insert admi Function**

```
app.post('/admi', jsonParser, function (req, res) {
  let adminfo = req.body.Adminfo;
  let loginfo = req.body.Loginfo;
  console.log(adminfo, loginfo);

  // Test input
  if (!adminfo || !adminfo.first_name || !adminfo.last_name || !adminfo.age || !adminfo.gender || !adminfo.phone || !adminfo.email) {
    return res.status(400).send({ error: true, message: 'Please provide complete administrator information' });
  }
  if (!loginfo || !loginfo.login_id || !loginfo.pass || !loginfo.login_role) {
    return res.status(400).send({ error: true, message: 'Please provide complete administrator login information' });
  }

  connection.query("INSERT INTO adm_information SET ?", adminfo, function (error, results) {
    if (error) {
      console.error("Error inserting data:", error);
      return res.status(500).send({ error: true, message: 'Error occurred while creating a new administrator' });
    }

    connection.query("INSERT INTO adm_login SET ?", loginfo, function (error, results) {
      if (error) {
        console.error("Error inserting data:", error);
        return res.status(500).send({ error: true, message: 'Error occurred while creating a new administrator login infor' });
      }
      return res.send({ error: false, message: 'New administrator has been created!' });
    });
  });
});
```

*Fig 35. Insert  Function*

This code defines a POST route named /admi to handle requests sent from the front-end to add new administrators. When the request is received, the server extracts the administrator information (adminfo) and login information (loginfo) from the body of the request, and then inserts this data into the database.

Parse the JSON data in the request body using jsonParser. Extract the administrator information (adminfo) and login information (loginfo) from the request body. Validate the input: make sure that both adminfo and loginfo are complete. If incomplete, an error response with HTTP status code 400 is returned. First insert the administrator information into the adm_information table. If an error occurs, return an error response with an HTTP status code of 500. Next, insert the login

information into the adm_login table. If an error occurs, an error response with HTTP status code 500 is returned. If the data is inserted successfully, a JSON response is returned with a success message.

**Update Admi function**

```
app.put('/admi/:id', jsonParser, function (req, res) {
  let adminfo = req.body.Adminfo;
  let loginfo = req.body.Loginfo;
  let a_uid = req.params.id;
  console.log(adminfo, loginfo, a_uid);
  console.log('Request body:', req.body);
  console.log('Request params:', req.params);
  if (!adminfo || !adminfo.first_name || !adminfo.last_name || !adminfo.age || !adminfo.gender || !adminfo.phone || !adminfo.email) {
    return res.status(400).send({ error: true, message: 'Please provide complete administrator information' });
  }
  if (!loginfo || !loginfo.login_id || !loginfo.pass || !loginfo.login_role) {
    return res.status(400).send({ error: true, message: 'Please provide complete administrator login information' });
  }

  connection.query("UPDATE adm_information SET ? WHERE uid = ?", [adminfo, a_uid], function (error, results) {
    if (error) {
      console.error("Error inserting data:", error);
      return res.status(500).send({ error: true, message: 'Error occurred while update administrator' });
    }

    connection.query("UPDATE adm_login SET ? WHERE login_id = ?", [loginfo, a_uid], function (error, results) {
      if (error) {
        console.error("Error inserting data:", error);
        return res.status(500).send({ error: true, message: 'Error occurred while update administrator login infor' });
      }
      return res.send({ error: false, message: 'Adm information has been updated!' });
    });
  });
});
```

*Fig 36. Update Function*

This code defines a PUT route named /admi/:id to handle requests sent from the front-end to update administrator information. When a request is received, the server extracts the administrator information (adminfo) and login information (loginfo) from the body of the request and updates the corresponding record in the database based on the administrator ID (a_uid) in the request parameters.

**Delete Admi Function**

```
// Delete user
// Testing Delete the admi DELETE url:http://localhost:5000/admi
// Case 1{
// "a_uid": "a456789"
// }
//Case 2{
// "a_uid": "a4567"
// }
app.delete("/admi/:id", jsonParser, function (req, res) {
  const admiID = req.params.id;

  if (!admiID) {
    return res.status(400).send({ error: true, message: 'Please provide admiID' });
  }

  connection.query("DELETE FROM adm_login WHERE login_id = ?", admiID, function (error, results) {
    if (error) {
      console.error("Error deleting data:", error);
      return res.status(500).send({ error: true, message: 'Error occurred while deleting administrator login: ' + error.message });
    }

    connection.query("DELETE FROM adm_information WHERE uid = ?", admiID, function (error, results) {
      if (error) {
        console.error("Error deleting data:", error);
        return res.status(500).send({ error: true, message: 'Error occurred while deleting administrator: ' + error.message });
      }
      return res.send({ error: false, message: 'Administrator information has been deleted!' });
    });
  });
});
```

*Fig 37. Delete  Function*

This code defines a DELETE route named /admi/:id that is used to handle requests sent from the front-end to delete administrator information. When a request is received, the server extracts the administrator ID (admiID) from the request parameters and removes the corresponding administrator information and login information from the database.

**Login Function**

```
//login
app.post('/login', jsonParser, (req, res) => {
  const loginId = req.body.loginId;
  const password = req.body.password;

  console.log("Trying to log in with:", loginId, password);
  connection.query('SELECT * FROM adm_login WHERE login_id = ? AND pass = ?', [loginId, password], (error, results) => {
    if (error) {
      console.error("Error while querying:", error);
      return res.status(500).send({ error: true, message: 'Error occurred while checking login information' });
    }
    console.log("Query results:", results);
    if (results.length > 0) {
      res.send({ error: false, message: 'Login successful', data: results[0] });
    } else {
      res.status(400).send({ error: true, message: 'Invalid login credentials' });
    }
  });
});

app.listen(port, ()=>{
    console.log(`Server is listening on port ${port}`);
});
```

*Fig 38. Login Function*

This code defines a POST route named /login that is used to handle user login requests. When a request is received, the server extracts the loginId and password from the request body, and then queries the database for the existence of the corresponding administrator login information.

**APIuse JavaScript**

```
var currencyDropdown = document.getElementById("currency-dropdown");

// Get the element used to display the price
var priceElement = document.getElementById("pd_price");
console.log("Price element value: " + priceElement.textContent);
// Get the button used to update the price
var currencyButton = document.getElementById("currency-button");

// Call the updatePrice function when the currency selection has changed
currencyDropdown.addEventListener("change", updatePrice);


function updatePrice() {
  // Get the selected target currency code
  var targetCurrency = currencyDropdown.value;
  console.log(targetCurrency);

  // Get the amount to be converted
  var amount = parseInt(priceElement.getAttribute("data-price"));
  //console.log(amount);

  // Constructing the URL of the API request
  var url = "https://api.apilayer.com/fixer/convert?to=" + targetCurrency +"&from=THB"+ "&amount=" + amount;
  var myHeaders = new Headers();
  myHeaders.append("apikey", "9jI20wSgO2xCvdiJxFAMIFmR64OhDq9q");
  var requestOptions = {
    method: 'GET',
    redirect: 'follow',
    headers: myHeaders,
    mode: 'cors' // Add this line to enable CORS
  };
  // Calling the API and updating prices
  fetch(url,requestOptions)
    .then(response => response.json())
    .then(data => {
      // Get the converted amount
      var convertedAmount = data.result;
      console.log(convertedAmount);
```

*Fig 39. API use JS code1*

```
      // Formatted display prices
      var formattedPrice = new Intl.NumberFormat('en-US', { style: 'currency', currency: targetCurrency }).format(convertedAmount);

      // Update the displayed price
      priceElement.textContent = formattedPrice;

      // Return Price Value
      return convertedAmount;
    })
    .catch(error => console.log('API error:', error));
}
```

*Fig 40. API use JS code2*

This code implements a currency conversion function. It first takes three DOM elements: a currency dropdown, an element to display the price, and a button to update the currency. When the currency drop-down menu changes, it calls a function called "updatePrice". This function first gets the code of the selected currency and the amount to be converted, then constructs an API request URL. it initiates the API request and updates the price using the fetch command with CORS added, then returns the converted amount. Finally, it formats the converted amount into the appropriate currency format and updates it to the page.

## Web Services Testing Results

Product Search page test

Test1:Testing search a product

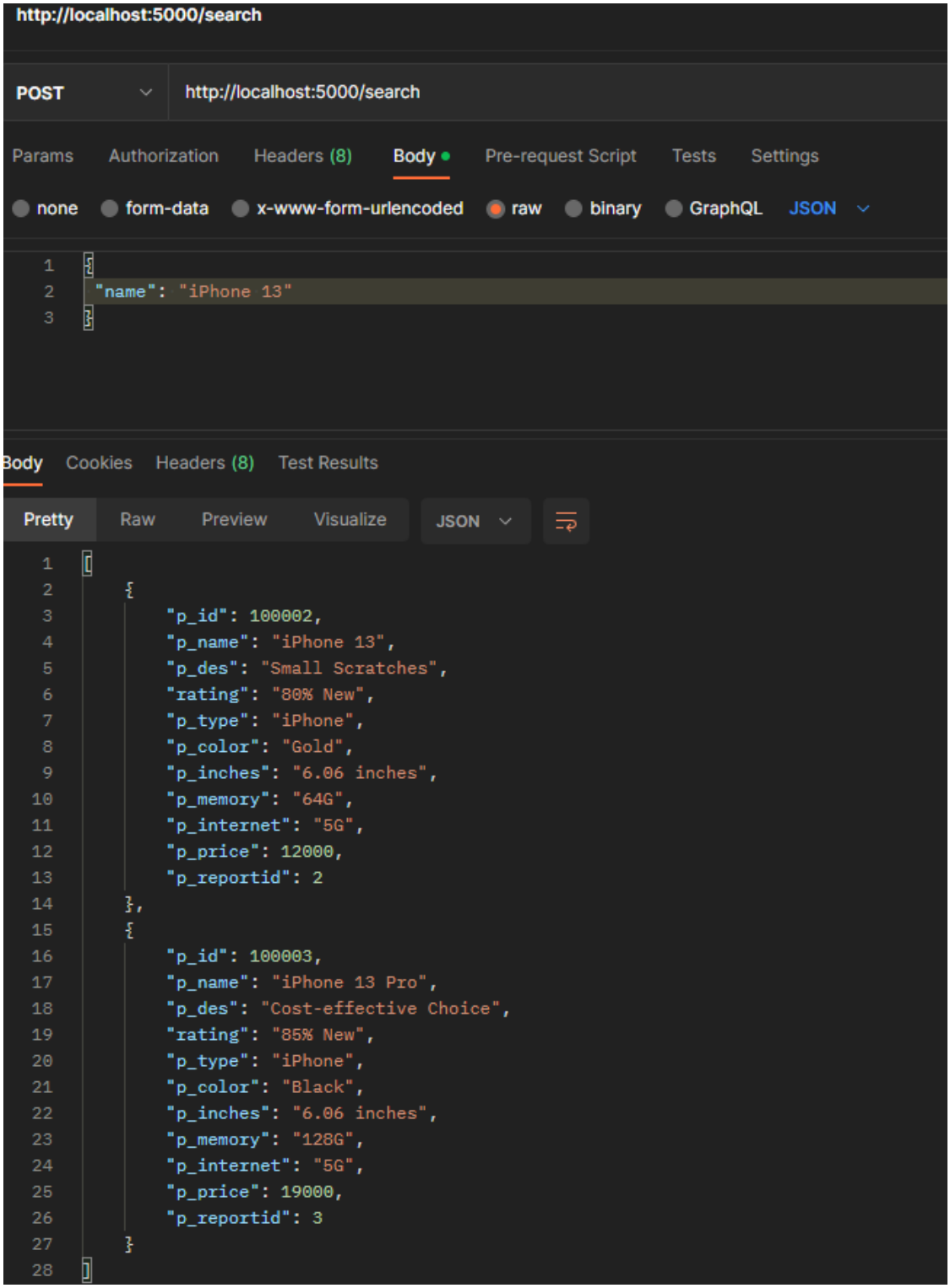



*Fig 41. Test to search product*

Test2:Return one Product

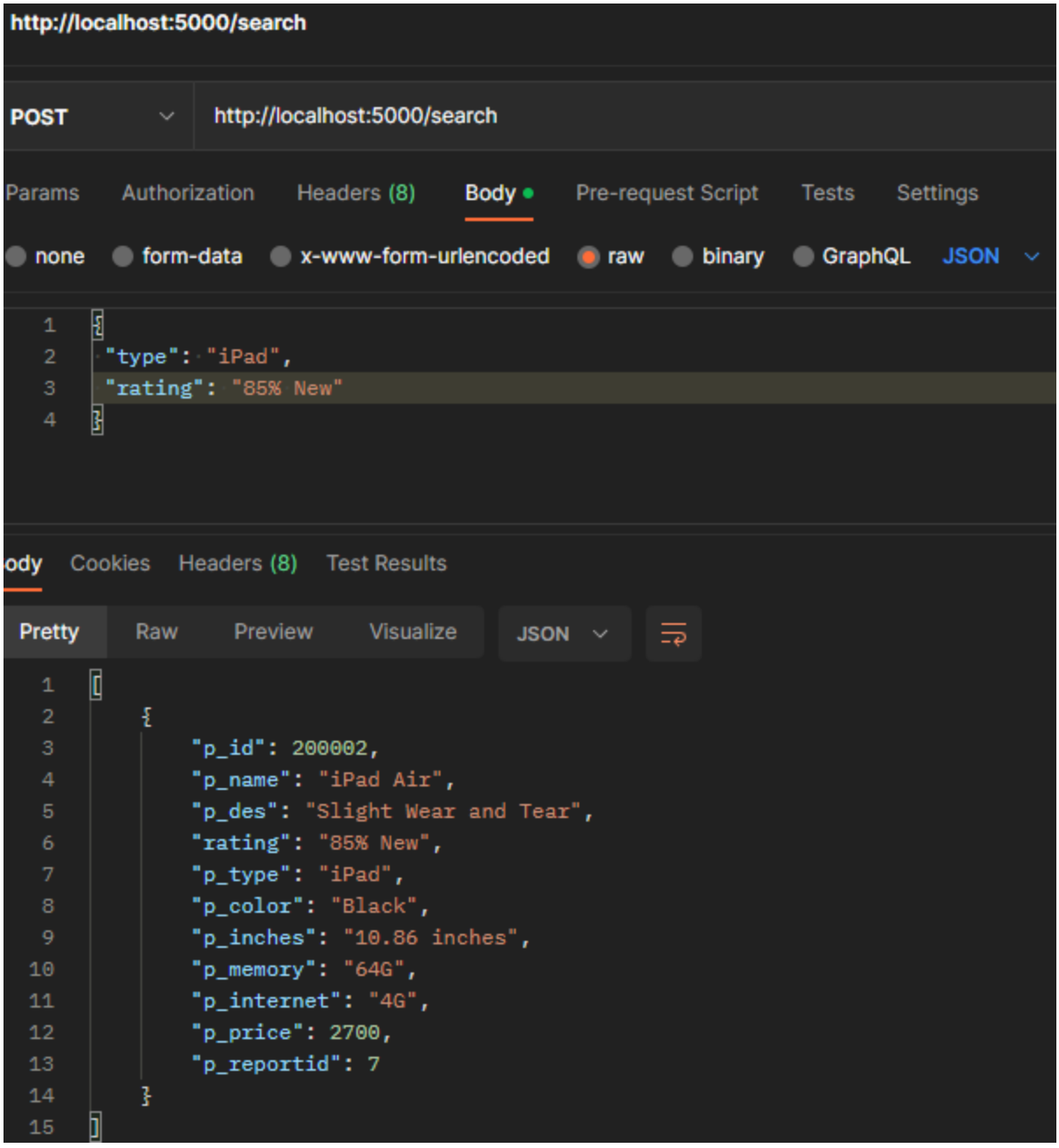



*Fig 42. Test to search product2*

Product management page test

Testing insert a product

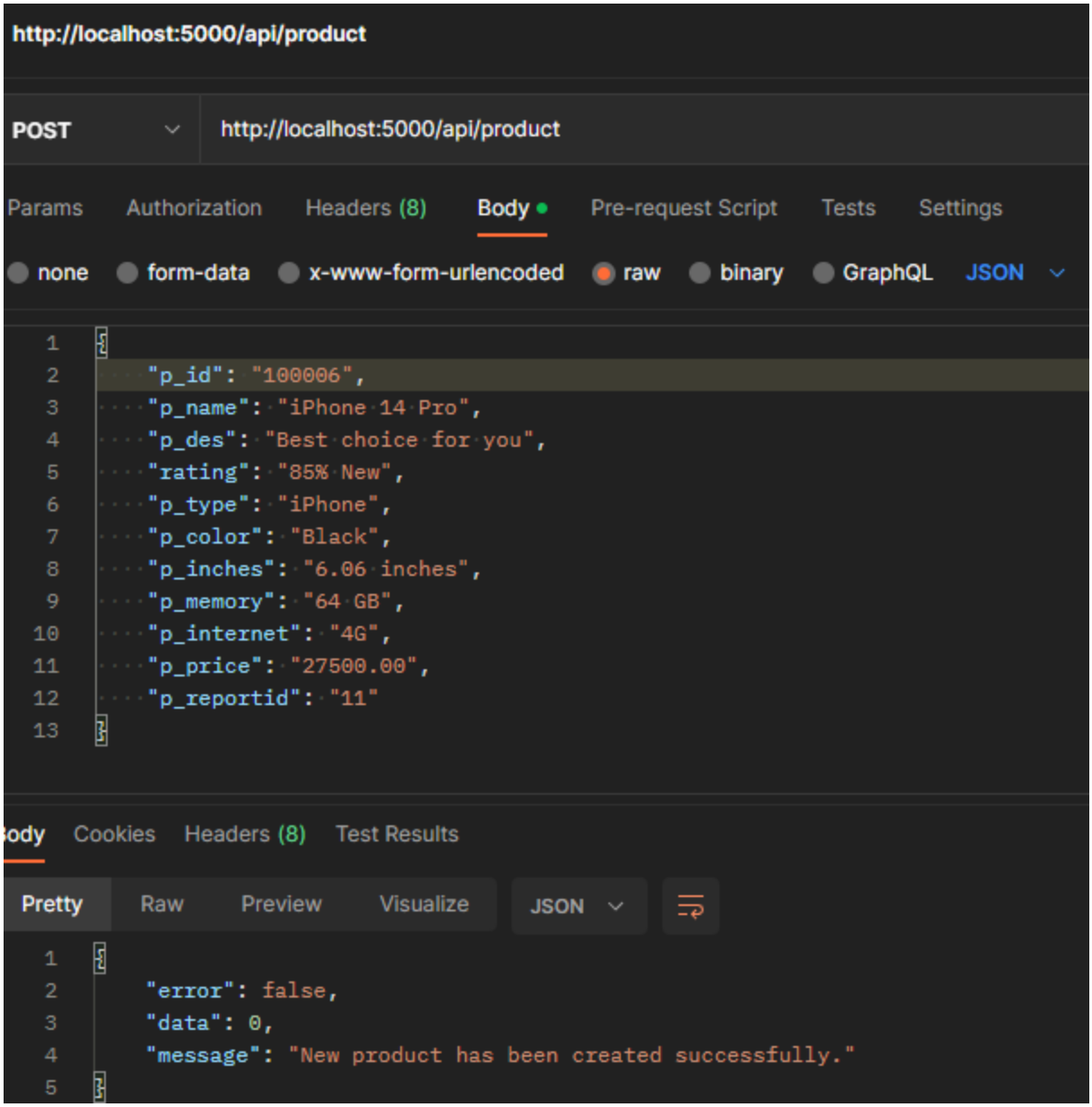



*Fig 43. Test to insert product*

Testing `(return "Error: read ECONNRESET")`

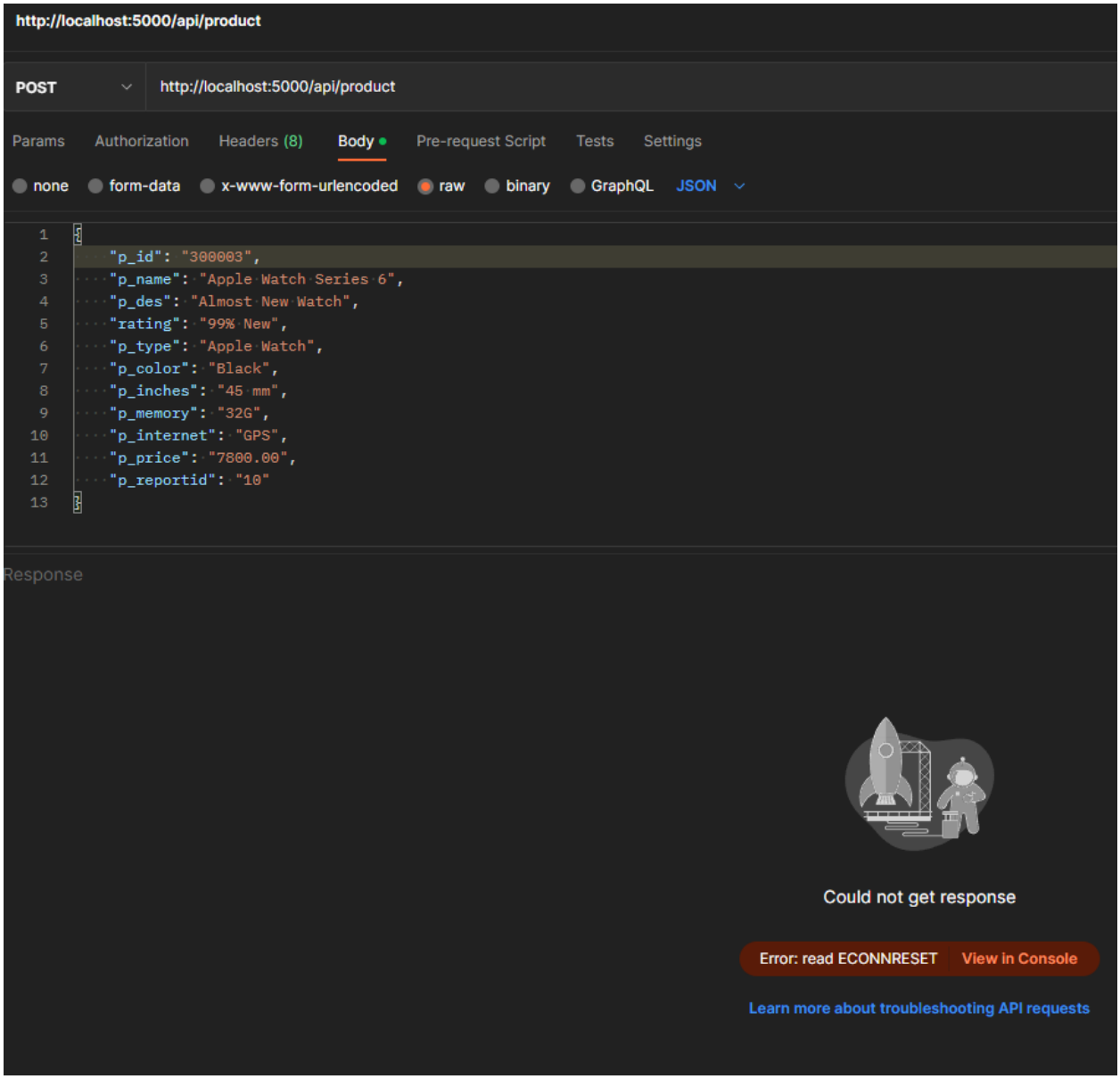



*Fig 44. Test insert product fail*

Testing update a product(add product)

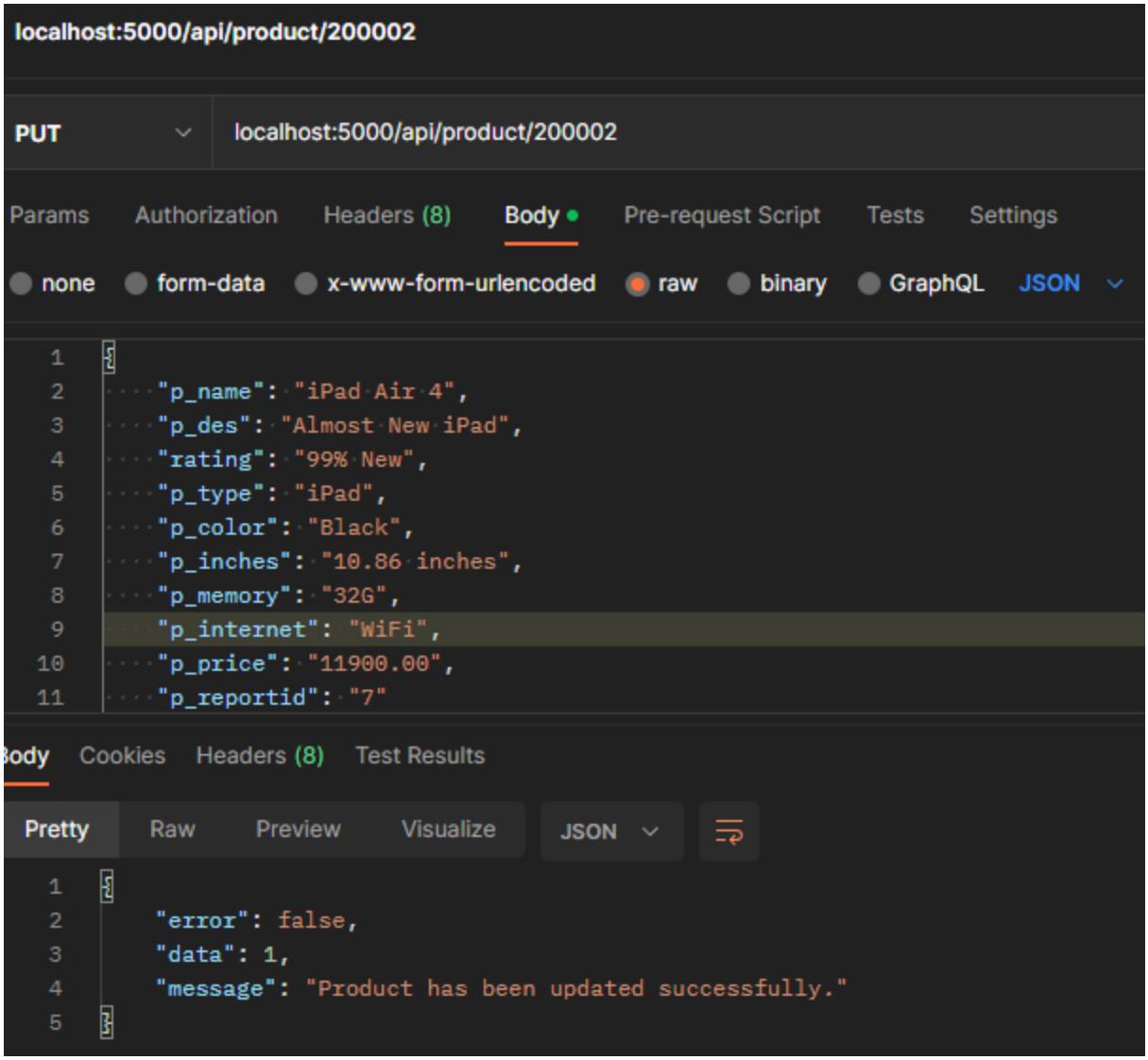



*Fig 45. Test to update product1*

Testing update a product(update product)

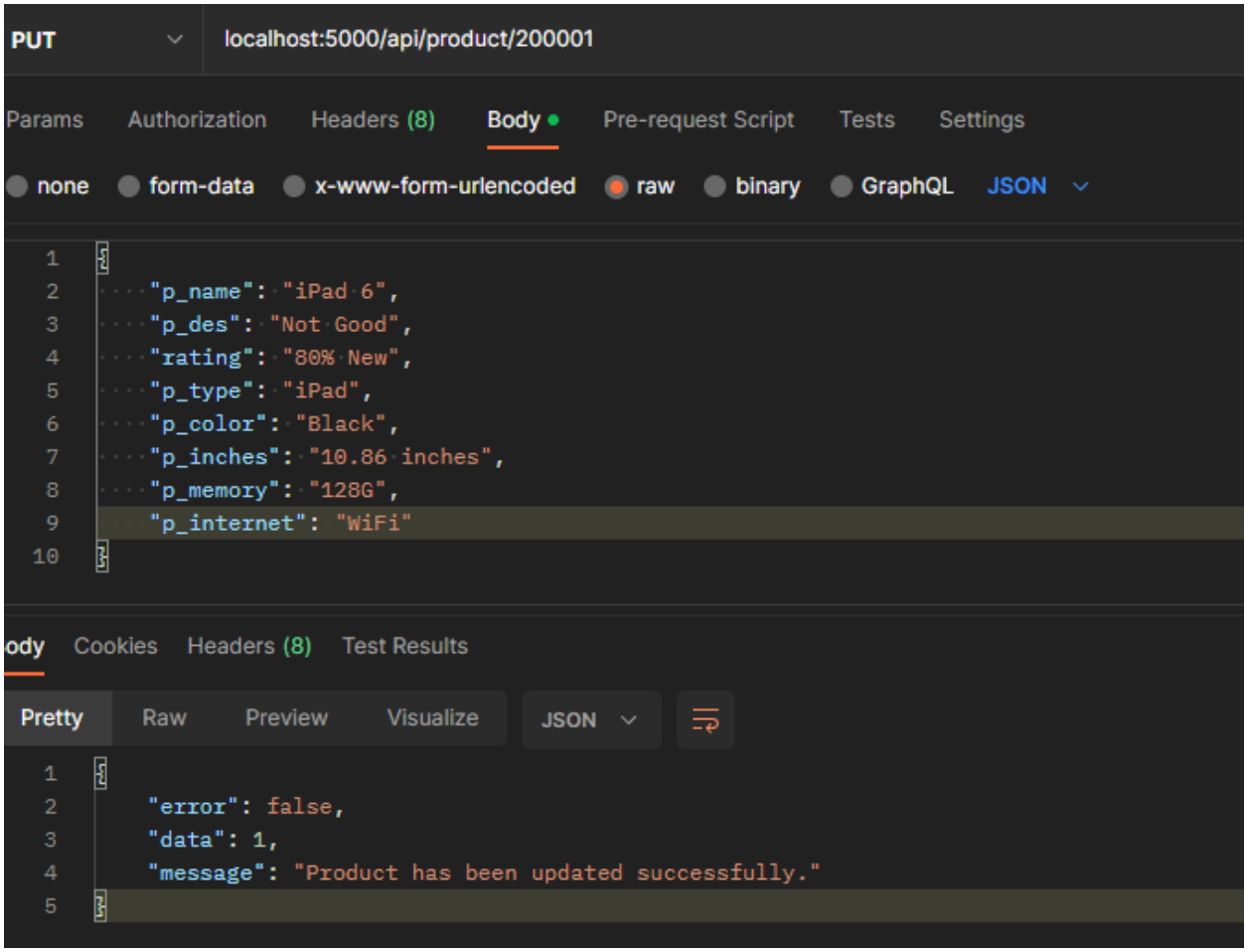



*Fig 46. Test to update product 2*

Testing for delete product

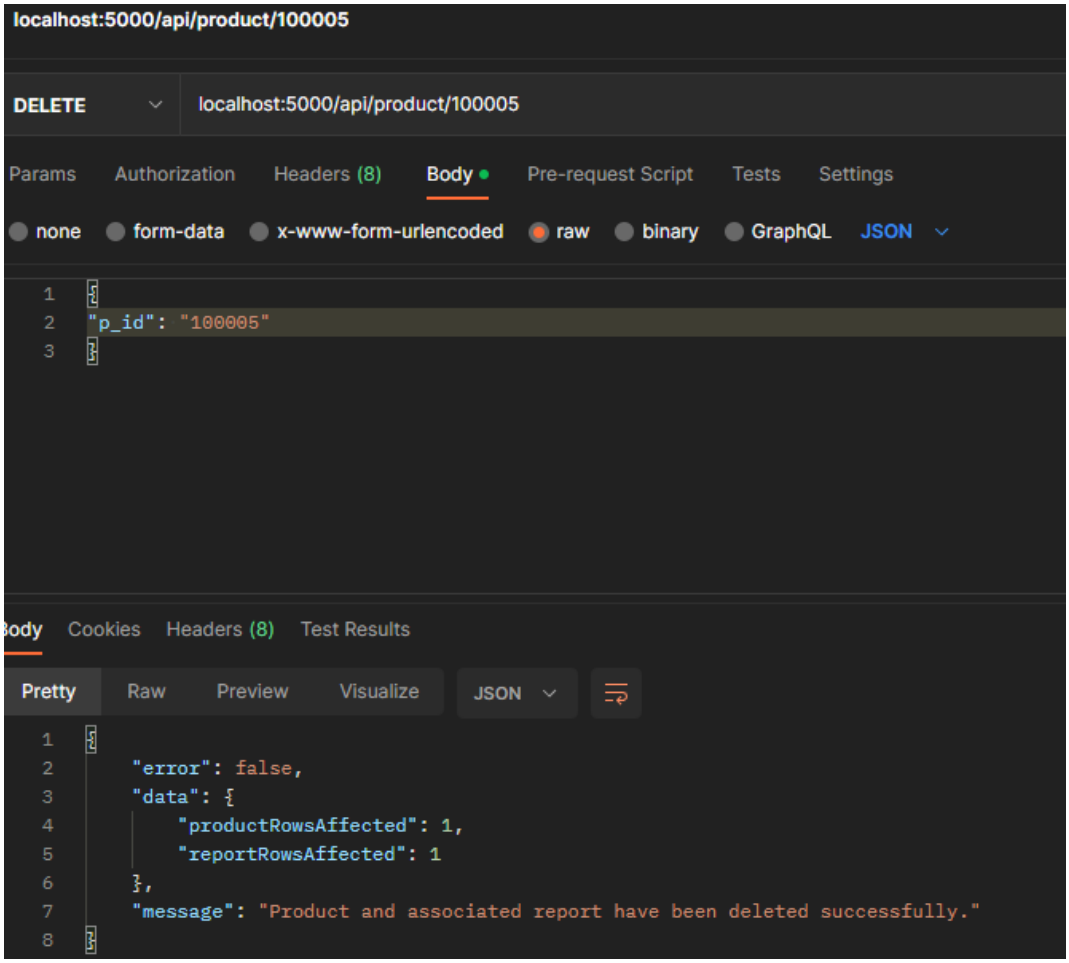



*Fig 47. Test to delet product 1*

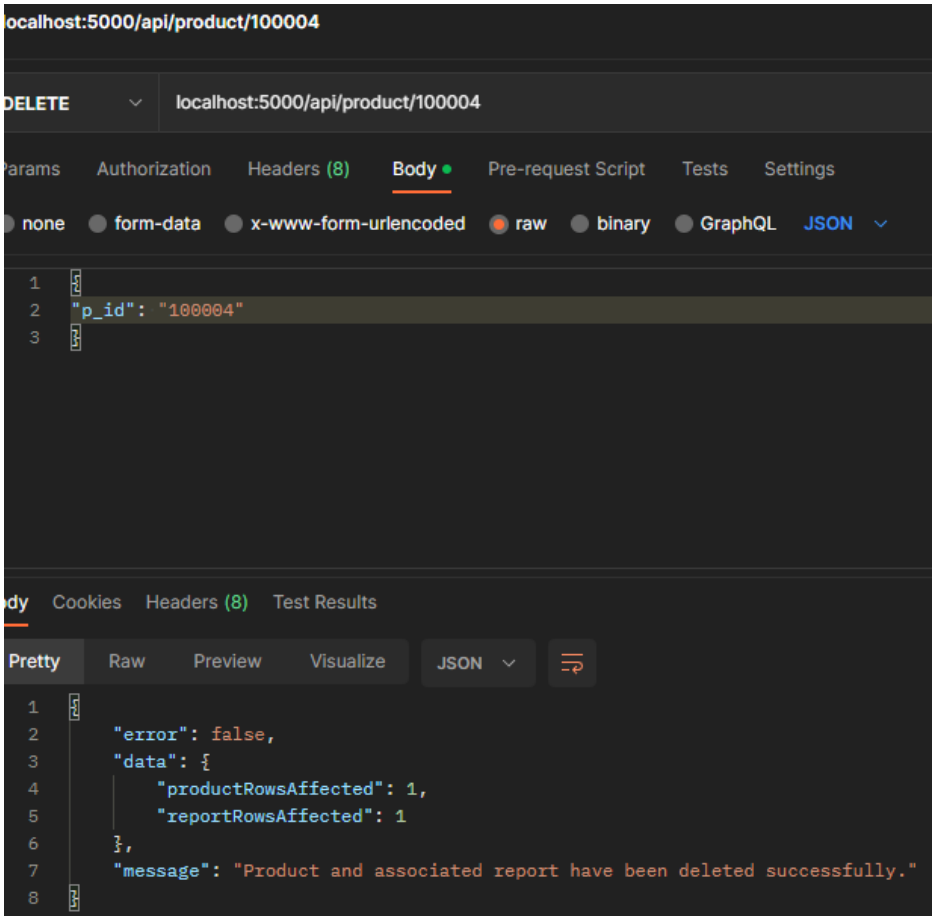



*Fig 48. Test to delete product 2*

Test for Administrator page

Test search Administer

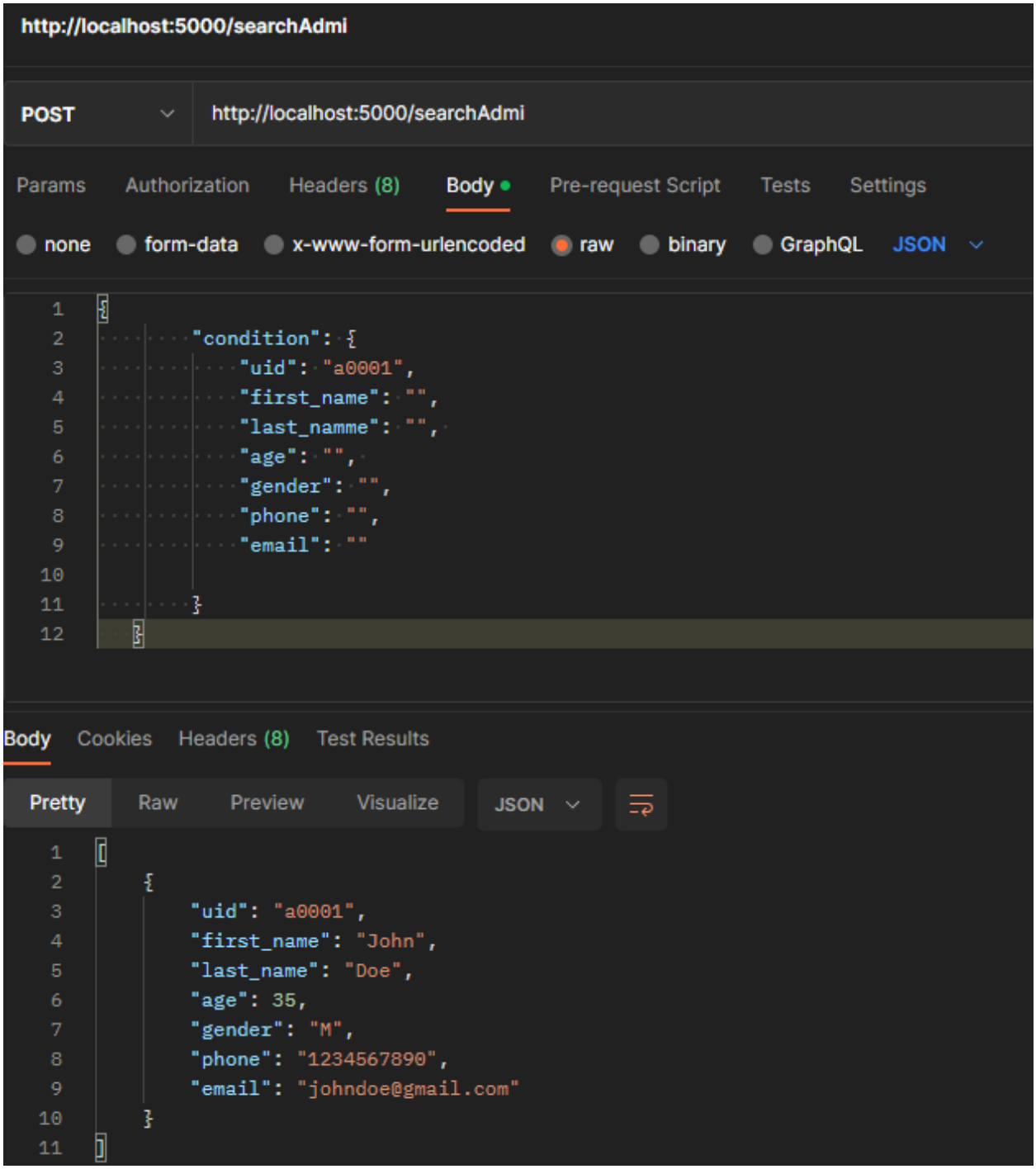



*Fig 49. Test to search Administrator 1*

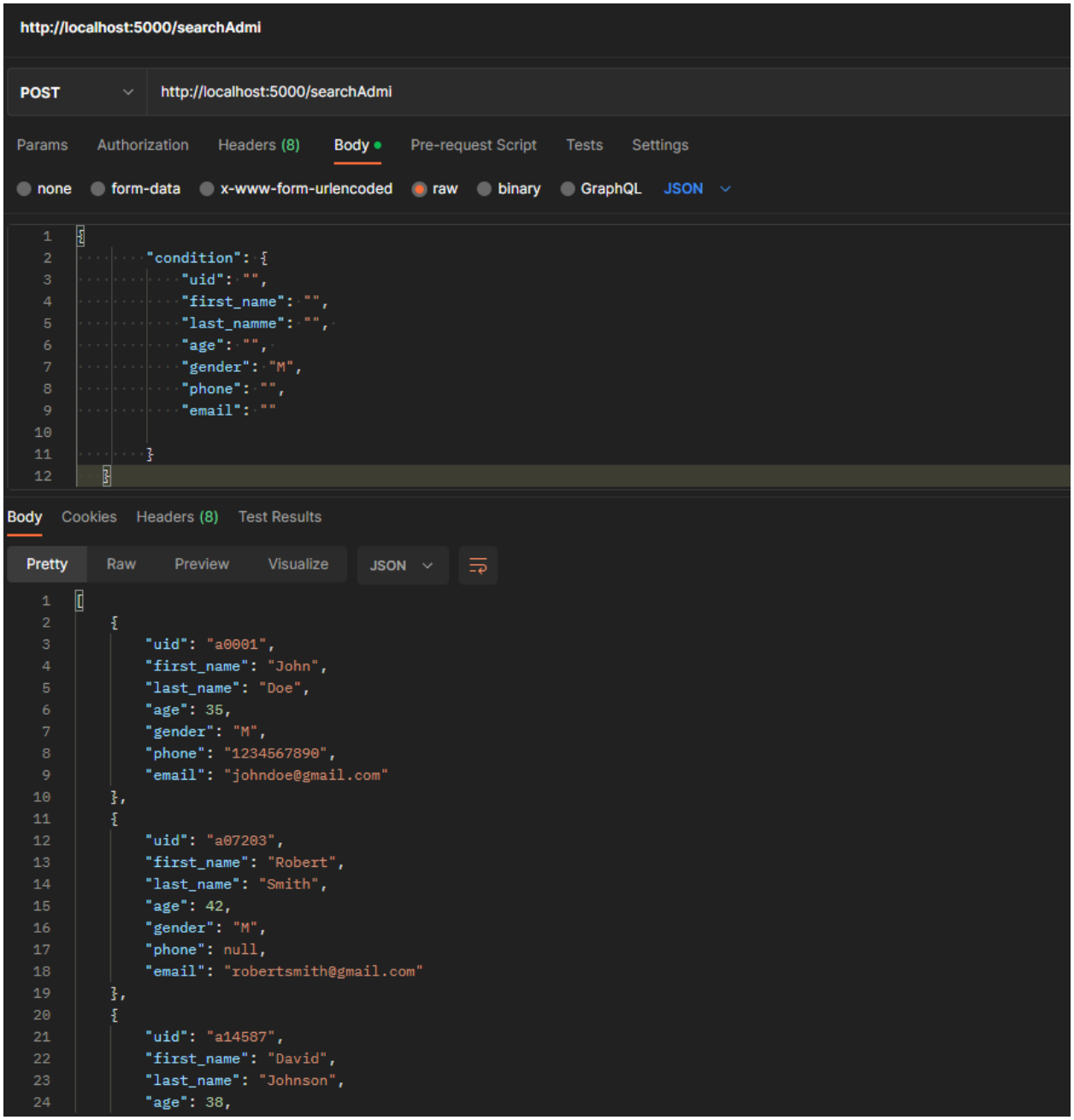



*Fig 50. Test to search Administrator 2*

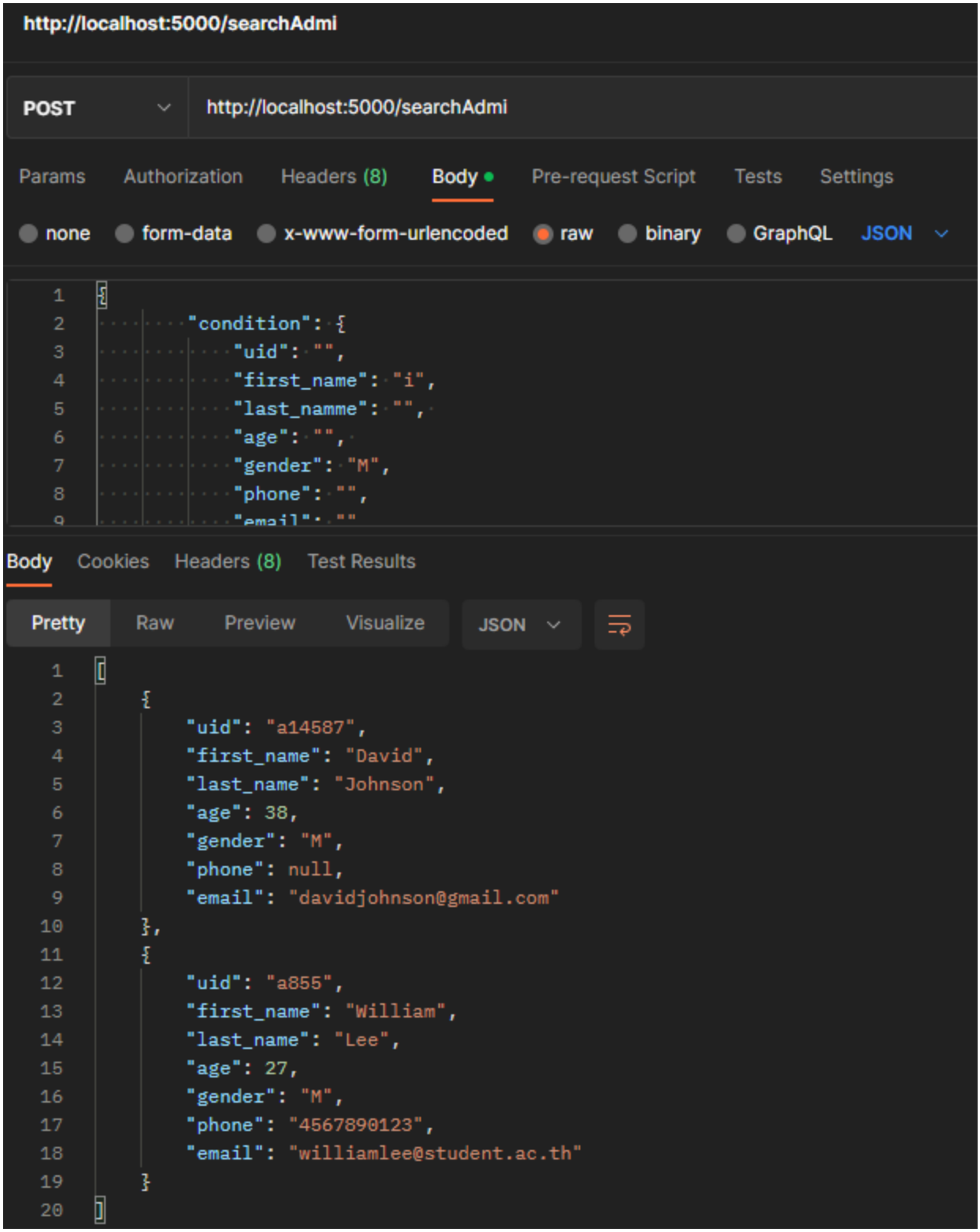



*Fig 51. Test to search Administrator 3*

Testing insert user

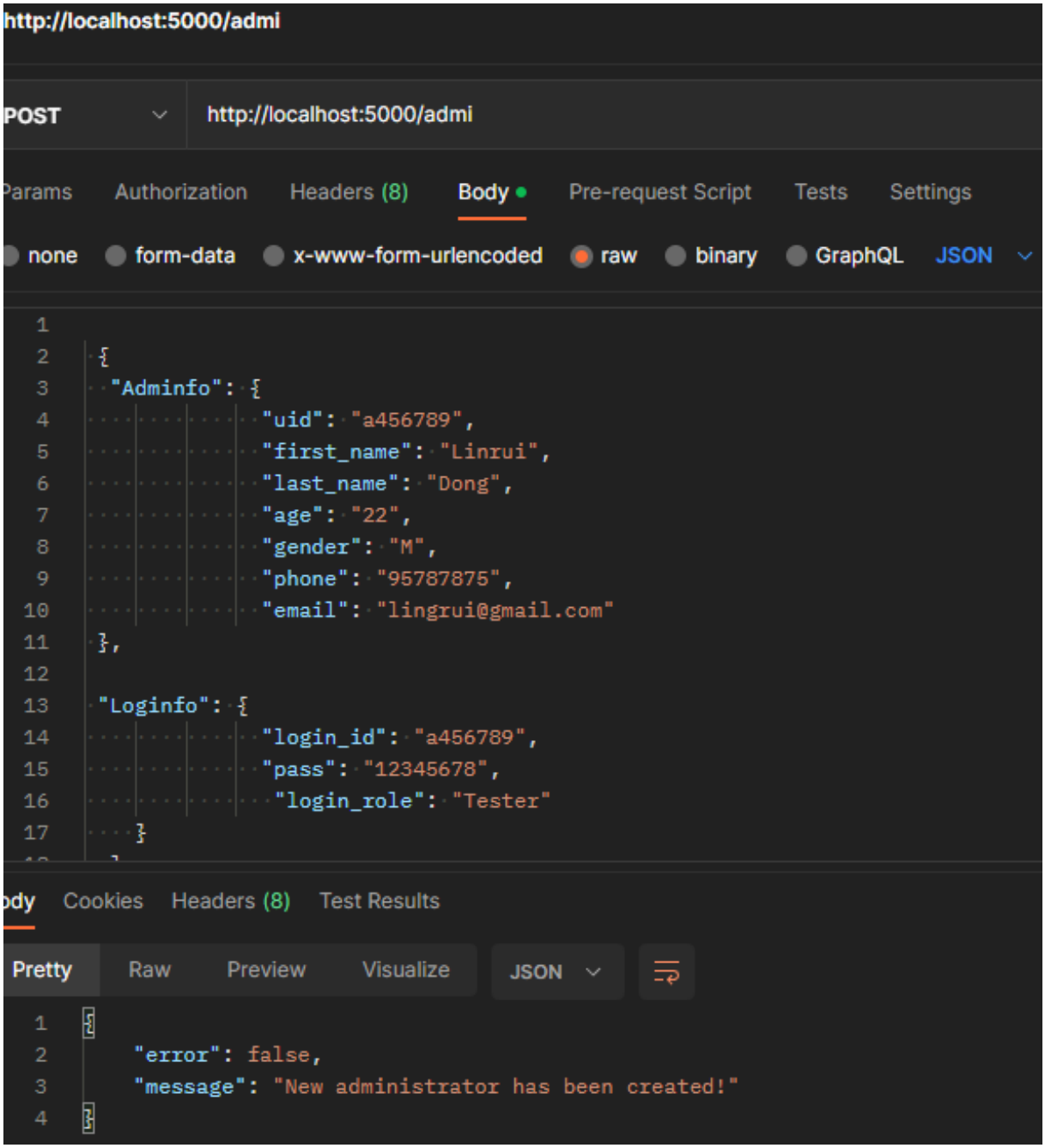



*Fig 52. Test to insert user 1*

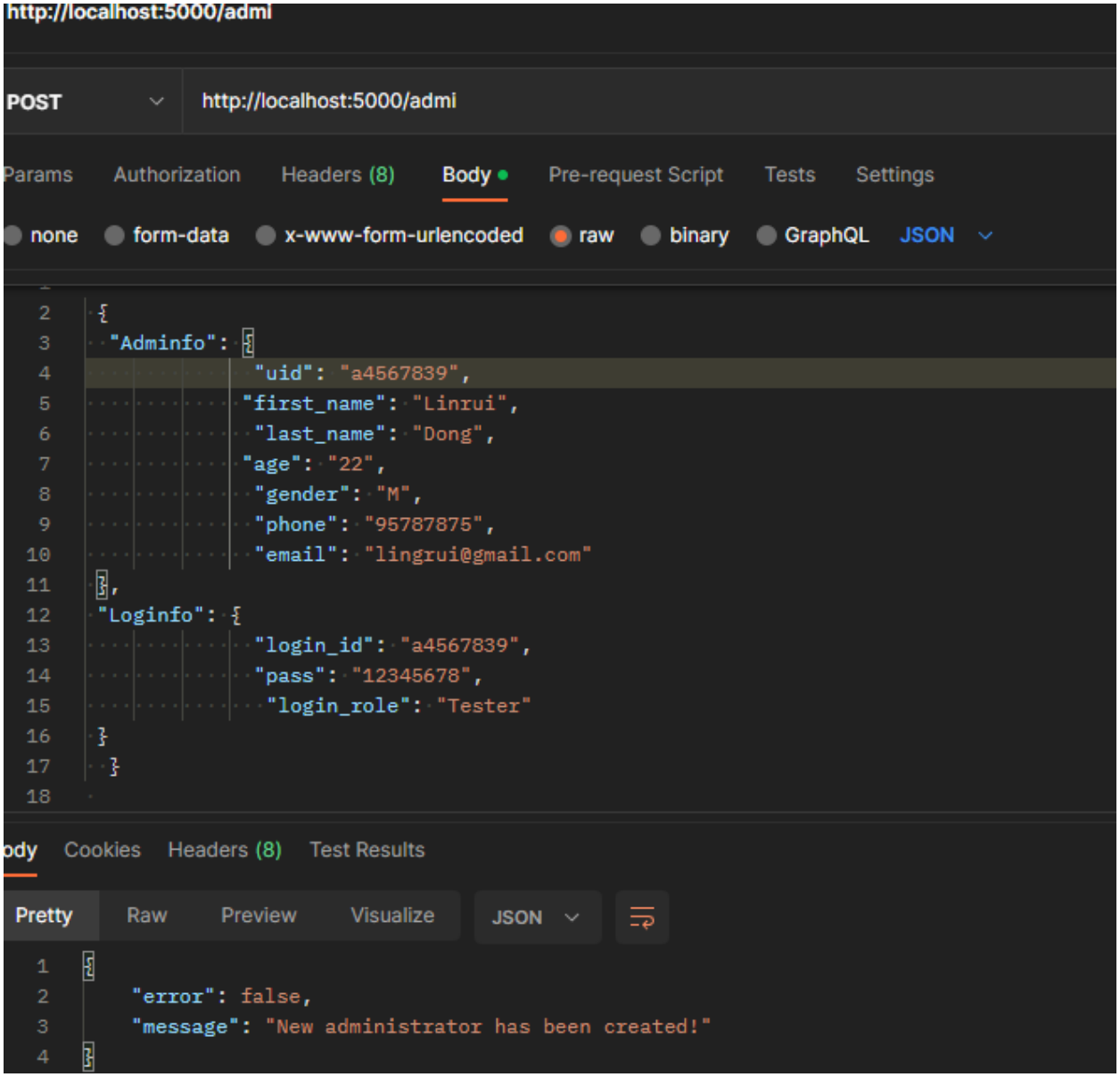



*Fig 53. Test to insert user 2*

Testing Update Administrator

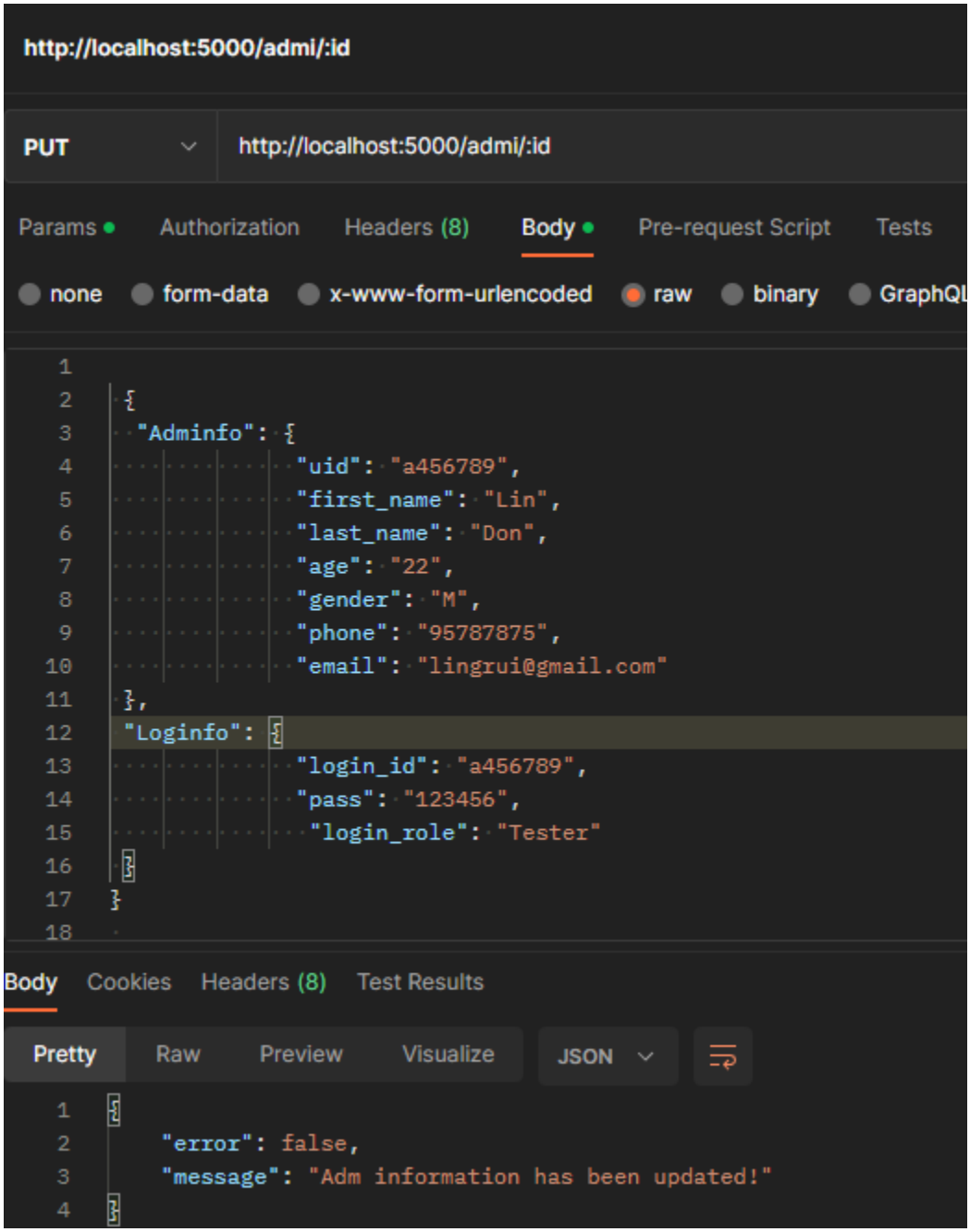



*Fig 54. Test to update Administrator 1*

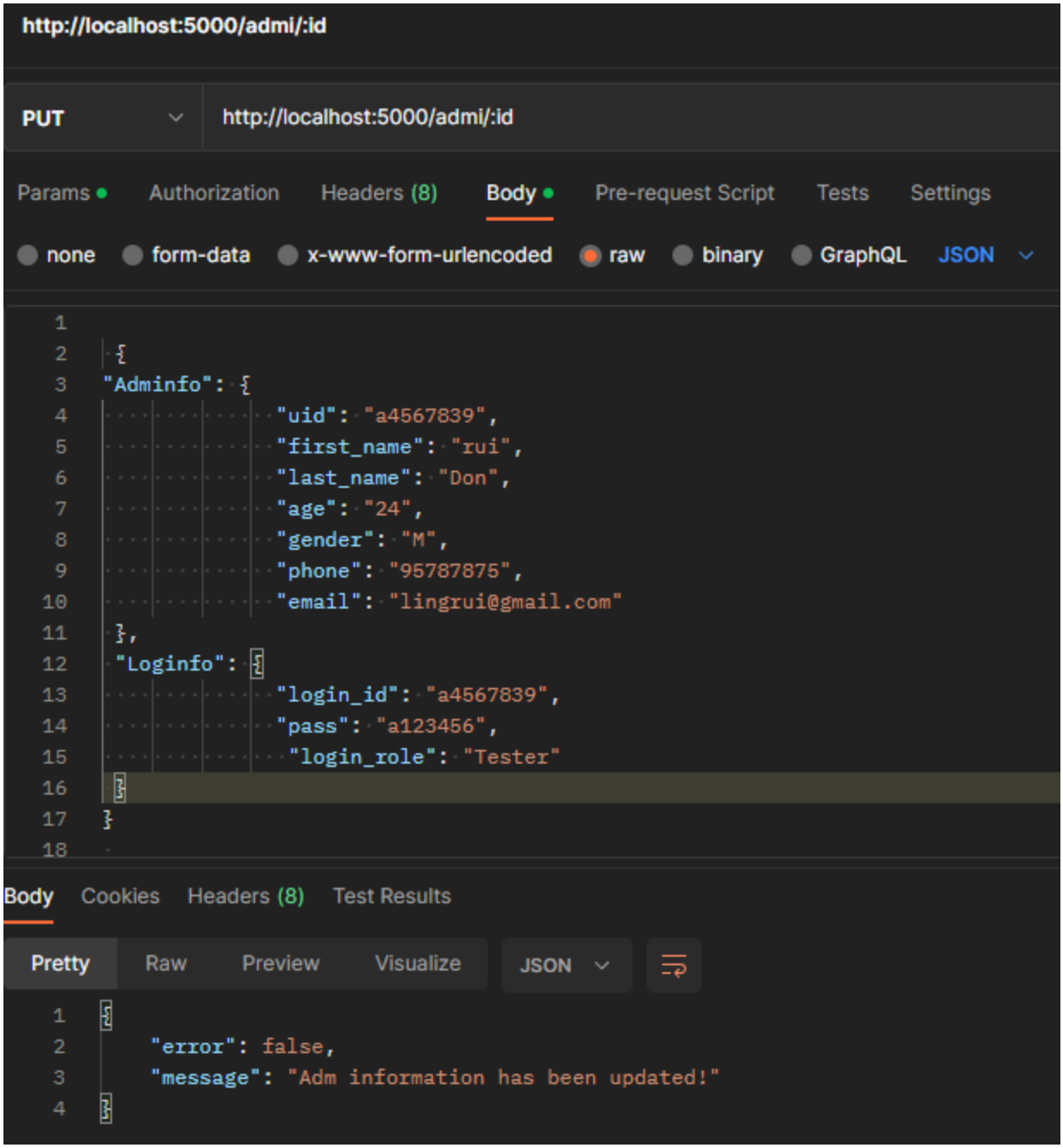



*Fig 55. Test to update Administrator 2*

Testing delete Administrator

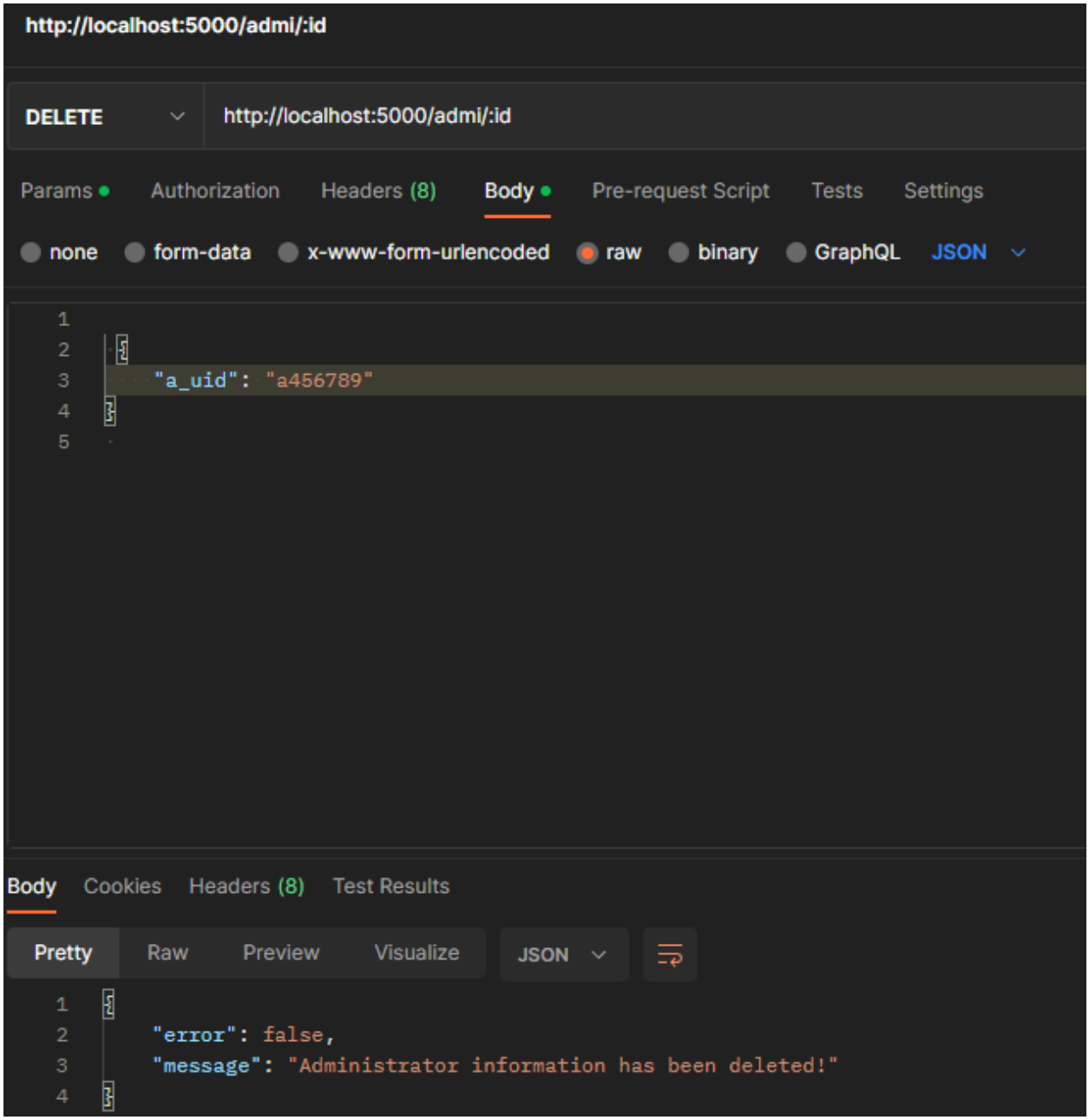



*Fig 56. Test to delete Administrator 1*

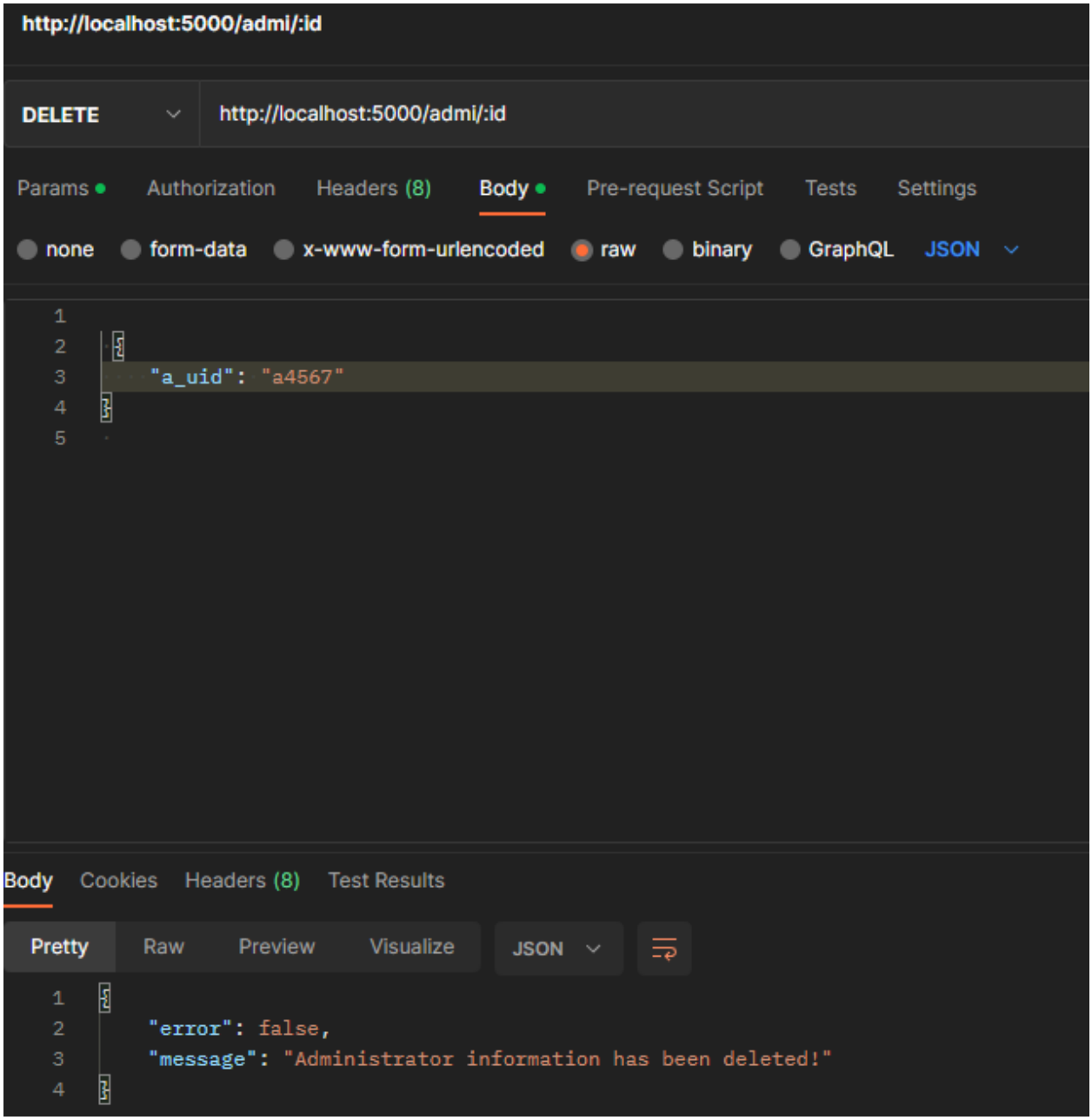



*Fig 57. Test to delete Administrator 2*

Testing login

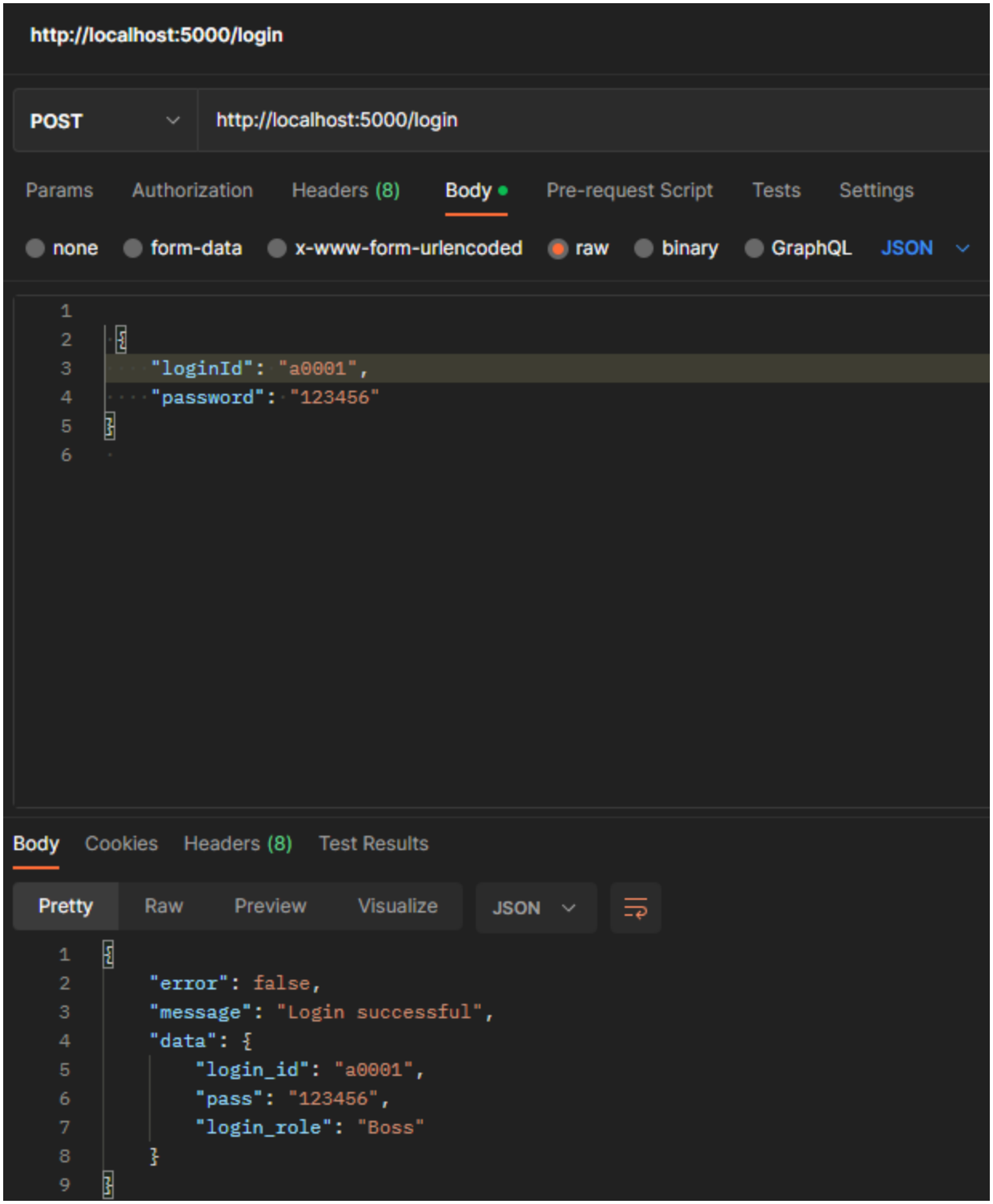



*Fig 58. Test to login*